



JAKARTA EE

Jakarta Concurrency

Jakarta Concurrency Team, <https://projects.eclipse.org/projects/ee4j.cu>

3.2, October 07, 2026: DRAFT

Table of Contents

Eclipse Foundation Specification License - v2.0	2
Disclaimers.....	3
Jakarta Concurrency Specification, Version 3.2	4
1. Introduction	5
1.1. Overview	5
1.2. Goals of this specification.....	5
1.3. Other Java Platform and Jakarta Specifications	5
1.4. Concurrency Utilities for Java EE Expert Group at the JCP	6
1.5. Document Conventions.....	6
2. Overview	7
2.1. Container-Managed vs. Unmanaged Threads.....	7
2.2. Application Integrity	7
2.3. Container Thread Context	8
2.3.1. Contextual Invocation Points.....	9
2.3.1.1. Flow Contextual Invocation Points.....	9
2.3.1.2. Optional Contextual Invocation Points	9
2.3.2. Contextual Objects and Tasks	10
2.3.2.1. Tasks and Jakarta Contexts and Dependency Injection (CDI)	10
2.4. Usage with Jakarta Connectors.....	11
2.5. Security.....	11
3. Managed Objects	12
3.1. ManagedExecutorService.....	12
3.1.1. Application Component Provider's Responsibilities	12
3.1.1.1. Usage Example	13
3.1.2. Application Assembler's Responsibilities.....	18
3.1.3. Deployer's Responsibilities	18
3.1.4. Jakarta EE Product Provider's Responsibilities.....	18
3.1.4.1. ManagedExecutorService Configuration Attributes	19
3.1.4.2. Configuration Examples	19
3.1.4.3. Default ManagedExecutorService	21
3.1.5. System Administrator's Responsibilities.....	22
3.1.6. Lifecycle	22
3.1.6.1. Jakarta EE Product Provider Requirements.....	23
3.1.7. Quality of Service	23
3.1.8. Transaction Management.....	23
3.1.8.1. Jakarta EE Product Provider Requirements.....	23
3.1.8.2. Application Component Provider's Requirements.....	24
3.2. ManagedScheduledExecutorService	25

3.2.1. Application Component Provider's Responsibilities	25
3.2.1.1. Usage Example	26
3.2.2. Application Assembler's Responsibilities	28
3.2.3. Deployer's Responsibilities	28
3.2.4. Jakarta EE Product Provider's Responsibilities	28
3.2.4.1. ManagedScheduledExecutorService Configuration Attributes	28
3.2.4.2. Configuration Examples	29
3.2.4.3. Default ManagedScheduledExecutorService	30
3.2.5. System Administrator's Responsibilities	30
3.2.6. Lifecycle	30
3.2.6.1. Jakarta EE Product Provider Requirements	31
3.2.7. Quality of Service	31
3.2.8. Transaction Management	31
3.2.8.1. Jakarta EE Product Provider Requirements	32
3.2.8.2. Application Component Provider's Requirements	32
3.3. ContextService	32
3.3.1. Application Component Provider's Responsibilities	33
3.3.1.1. Usage Example	34
3.3.2. Application Assembler's Responsibilities	37
3.3.3. Deployer's Responsibilities	37
3.3.4. Jakarta EE Product Provider's Responsibilities	37
3.3.4.1. ContextService Configuration Attributes	38
3.3.4.2. Configuration Examples	38
3.3.4.3. Default ContextService	39
3.3.5. Transaction Management	39
3.3.5.1. Jakarta EE Product Provider Requirements	40
3.3.5.2. Application Component Provider's Requirements	40
3.4. ManagedThreadFactory	41
3.4.1. Application Component Provider's Responsibilities	41
3.4.1.1. Usage Example	42
3.4.2. Application Assembler's Responsibilities	44
3.4.3. Deployer's Responsibilities	44
3.4.4. Jakarta EE Product Provider's Responsibilities	44
3.4.4.1. ManagedThreadFactory Configuration Attributes	45
3.4.4.2. Configuration Examples	45
3.4.4.3. Default ManagedThreadFactory	46
3.4.5. System Administrator's Responsibilities	46
3.4.6. Transaction Management	46
3.4.6.1. Jakarta EE Product Provider Requirements	47
3.4.6.2. Application Component Provider's Requirements	47
3.5. Beans for Managed Objects	47

3.5.1. Qualifiers Example	48
3.5.1.1. Example Qualifier Annotation Class	48
3.5.1.2. Example usage of Qualifier Annotation	48
4. Thread Context Providers	50
4.1. Responsibilities and Requirements	50
4.1.1. Third-Party Context Provider's Requirements	50
4.1.2. Jakarta EE Product Provider's Requirements	50
4.2. Usage Examples	51
4.2.1. Custom Thread Context Example	51
4.2.1.1. Example of Custom ThreadContextProvider	51
4.2.1.2. Example of Custom ThreadContextSnapshot and ThreadContextRestorer	52
4.2.1.3. ServiceLoader Entry	52
4.2.1.4. Usage of the Custom Context Type from a Servlet	53
5. Asynchronous Methods	54
5.1. Scheduled Asynchronous Methods	54
5.2. Responsibilities and Requirements	54
5.2.1. Application Component Provider's Responsibilities	54
5.2.1.1. Usage Example	54
5.2.2. Jakarta EE Product Provider's Responsibilities	56
5.3. Transaction Management	56
6. Scheduled Methods	58
6.1. Skipped Invocation	58
6.2. Lifecycle	58
6.3. Responsibilities and Requirements	58
6.3.1. Application Component Provider's Responsibilities	58
6.3.1.1. Usage Example	58
6.3.2. Jakarta EE Product Provider's Responsibilities	59
6.4. Transaction Management	59

Specification: Jakarta Concurrency

Version: 3.2

Status: DRAFT

Release: October 07, 2026

Eclipse Foundation Specification License - v2.0

This license (the “License”) governs the use, modification and distribution of a specification generated pursuant to the Eclipse Foundation Specification Process.

By using the file or document that incorporates this License (the “Document”), you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions.

Permission to copy and distribute the contents of the Document, in any medium for any purpose, and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the Document that you copy or distribute:

- link or URL to the Document
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: “Copyright © [date-of-document] Eclipse Foundation AISBL [URL to this license] This document includes material copied from [title and URL of the Eclipse Foundation specification document].”

In addition to the license granted above, you are granted the right to create modifications or derivatives of the Document:

- in software that implements the Document, in supporting materials accompanying such software, and in documentation of such software (collectively the “Works”), such Works to be made available under terms of your choice PROVIDED that all such Works include a notice of the form: “This [software/document (choose as applicable)] includes material derived from [title and URL of the Eclipse Foundation specification document].”; and
- in materials that demonstrate or describe conformance with the Document PROVIDED that all such materials include a notice of the form: “This material demonstrates conformance with the [title and URL of the Eclipse Foundation specification document].”

NOTWITHSTANDING THE FOREGOING, the creation or publication of derivative works of the Document or portions of the Document for use as a specification or standard is expressly prohibited.

Disclaimers

THE DOCUMENT IS PROVIDED "AS IS," AND TO THE EXTENT PERMITTED BY APPLICABLE LAW THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION AISBL MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

TO THE EXTENT PERMITTED BY APPLICABLE LAW THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION AISBL WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation AISBL may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission.

Jakarta Concurrency Specification, Version 3.2

Copyright (c) 2013, 2026 Oracle and/or its affiliates. All rights reserved.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Chapter 1. Introduction

1.1. Overview

Jakarta™ Platform, Enterprise Edition (Jakarta EE) server containers such as the enterprise bean or web component container do not recommend using common Java SE concurrency APIs such as `java.lang.Thread` or `java.util.Timer` directly.

The server containers provide runtime support for Jakarta EE application components (such as Jakarta Servlets and Jakarta Enterprise Beans). They provide a layer between application component code and platform services and resources. All application component code is run on a thread managed by a container and each container typically expects all access to container-supplied objects to occur on the same thread.

It is because of this behavior that application components are typically unable to reliably use other Jakarta EE platform services from a thread that is not managed by the container.

Jakarta EE Product Providers (see chapter 2.12 of the Jakarta EE 12 Specification) also discourage the use of resources in a non-managed way, because it can potentially undermine the enterprise features that the platform is designed to provide such as availability, security, and reliability and scalability.

This specification provides a simple, standardized API for using concurrency from Jakarta EE application components without compromising the integrity of the container while still preserving the fundamental benefits of the Jakarta EE platform.

1.2. Goals of this specification

This specification was developed with the following goals in mind:

- Utilize existing applicable Jakarta EE platform services. Provide a simple yet flexible API for application component providers to design applications using concurrency design principles.
- Allow Java SE developers a simple migration path to the Jakarta EE platform by providing consistency between the Java SE and Jakarta EE platforms.
- Allow application component providers to easily add concurrency to existing Jakarta EE applications.
- Support simple (common) and advanced concurrency patterns without sacrificing usability.

1.3. Other Java Platform and Jakarta Specifications

The following Java Platform and Jakarta specifications are referenced in this document:

- Concurrency Utilities Specification (JSR-166)
- Jakarta Contexts and Dependency Injection
- Jakarta Connectors

- Java Platform Standard Edition
- Jakarta Management
- Java Naming and Directory Interface™
- Jakarta Transactions
- Jakarta Transaction Service
- JDBC™ API
- Jakarta Messaging
- Jakarta Enterprise Beans

1.4. Concurrency Utilities for Java EE Expert Group at the JCP

The original Java EE specification was the result of the collaborative work of the members of the Concurrency Utilities for Java EE Expert Group. The expert group includes the following members: Adam Bien, Marius Bogoevici (RedHat), Cyril Bouteille, Andrew Evers, Anthony Lai (Oracle), Doug Lea, David Lloyd (RedHat), Naresh Revanuru (Oracle), Fred Rowe (IBM Corporation), and Marina Vatkina (Oracle).

We would also like to thank former expert group members for their contribution to this specification, including Jarek Gawor (Apache Software Foundation), Chris D. Johnson (IBM Corporation), Billy Newport (IBM Corporation), Stephan Zachwiega (BEA Systems), Cameron Purdy (Tangosol), Gene Gleyzer (Tangosol), and Pierre VignJras.

1.5. Document Conventions

The regular Times font is used for information that is prescriptive to this specification.

The italic Times font is used for paragraphs that contain descriptive information, such as notes describing typical use, or notes clarifying the text with prescriptive specification.

The Courier font is used for code examples.

Chapter 2. Overview

The focus of this specification is on providing asynchronous capabilities to Jakarta EE application components. This is largely achieved through extending the Concurrency Utilities API developed under JSR-166 and found in Java Platform, Standard Edition (Java SE) in the `java.util.concurrent` package.

The Java SE concurrency utilities provide an API that can be extended to support the majority of the goals defined in section 1.2. Application developers familiar with this API in the Java SE platform can leverage existing code libraries and usage patterns with little modification.

This specification has several aspects:

- Definition and usage of centralized, manageable `java.util.concurrent.ExecutorService` objects in a Jakarta EE application server.
- Usage of Java SE Concurrency Utilities in a Jakarta EE application.
- Propagation of the Jakarta EE container's runtime contextual information to other threads.
- Managing and monitoring the lifecycle of asynchronous operations in a Jakarta EE Application Component.
- Preserving application integrity.

2.1. Container-Managed vs. Unmanaged Threads

Jakarta EE application servers require resource management in order to centralize administration and protect application components from consuming unneeded resources. This can be achieved through the pooling of resources and managing a resource's lifecycle. Using Java SE concurrency utilities such as the `java.util.concurrent` API, `java.lang.Thread` and `java.util.Timer` in a server application component such as a servlet or Jakarta Enterprise Bean are problematic since the container and server have no knowledge of these resources.

By extending the `java.util.concurrent` API, application servers and Jakarta EE containers can become aware of the resources that are used and provide the proper execution context for the asynchronous operations.

This is largely achieved by providing managed versions of the predominant `java.util.concurrent.ExecutorService` interfaces.

2.2. Application Integrity

Managed environments allow applications to coexist without causing harm to the overall system and isolate application components from one another. Administrators can adjust deployment and runtime settings to provide different qualities of service, provisioning of resources, scheduling of tasks, etc. Jakarta EE containers also provide runtime context services to the application component. When using concurrency utilities such as those in `java.util.concurrent`, these context services need to be available.

2.3. Container Thread Context

Jakarta EE depends on various context information to be available on the thread when interacting with other Jakarta EE services such as JDBC data sources, Jakarta Messaging providers and Jakarta Enterprise Beans. When using Jakarta EE services from a non-container thread, the following behaviors are required:

- Saving the application component thread's container context.
- Identifying which container contexts to save and propagate.
- Applying a container context to the current thread.
- Restoring a thread's original context.

The types of contexts to be propagated from a contextualizing application component include JNDI naming context, classloader, and security information. Containers must support propagation of these context types. In addition, containers can choose to support propagation of other types of context.

The relationships between the various Jakarta EE architectural elements, containers and concurrency constructs are shown in Figure 2-1.

Containers (represented here in a single rectangle) provide environments for application components to safely interact with Jakarta EE Standard Services (represented in the rectangles directly below the Enterprise Bean/Web Container rectangle). Four new concurrency services (represented by four orange horizontal rectangles) allow application components and Jakarta EE Standard Services to run asynchronous tasks without violating container contracts.

The arrows in the diagram illustrate various flows from one part of the Jakarta EE platform to another.

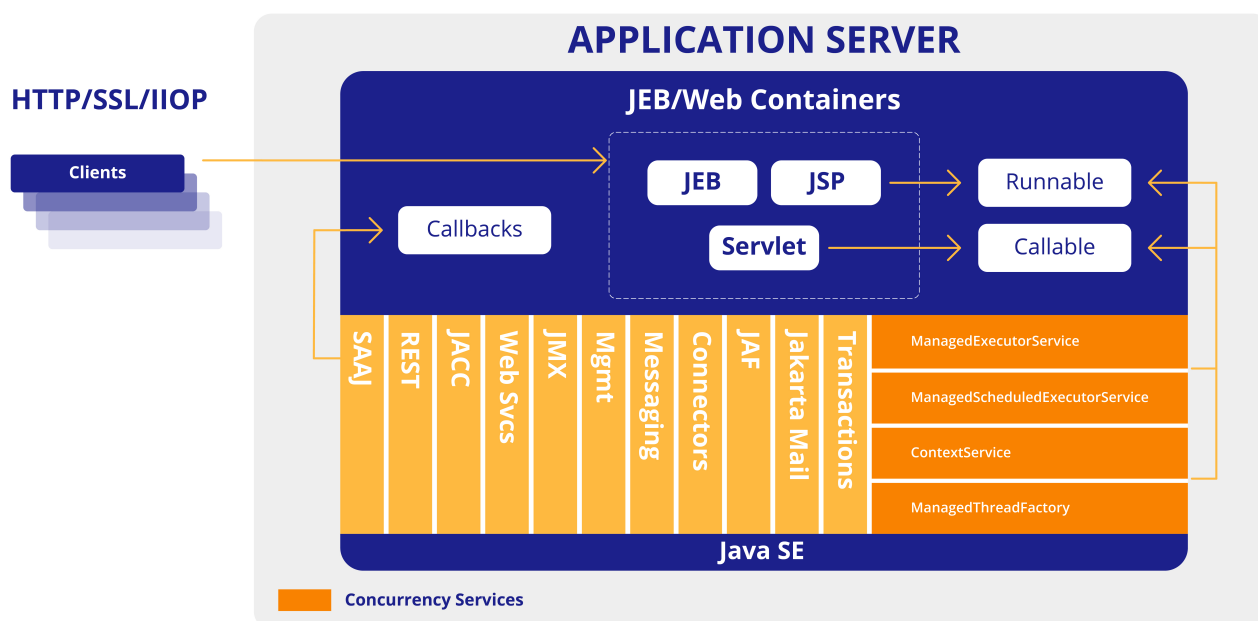


Figure 2-1 Concurrency Utilities for Jakarta EE Architecture Diagram

2.3.1. Contextual Invocation Points

Container context and management constructs are propagated to component business logic at runtime using various invocation points on well-known interfaces. These invocation points or callback methods, hereby known as "tasks" will be referred to throughout the specification:

- `java.util.concurrent.Callable`
 - `call()`
- `java.lang.Runnable`
 - `run()`
- `java.util.function.BiConsumer`
 - `accept(T, U)`
- `java.util.function.BiFunction`
 - `apply(T, U)`
- `java.util.function.Consumer`
 - `accept(T)`
- `java.util.function.Function`
 - `apply(T)`
- `java.util.function.Supplier`
 - `get()`

2.3.1.1. Flow Contextual Invocation Points

The `java.util.concurrent.Flow` class provides interfaces for establishing flow-controlled components. Container context and management constructs are also propagated to methods on the `java.util.concurrent.Flow.Subscriber` interface, including the inherited methods in `java.util.concurrent.Flow.Processor`.

2.3.1.2. Optional Contextual Invocation Points

The following callback methods run with unspecified context by default, but may be configured as contextual invocation points if desired:

- `jakarta.enterprise.concurrent.ManagedTaskListener`
 - `taskAborted()`
 - `taskSubmitted()`
 - `taskStarting()`
- `jakarta.enterprise.concurrent.Trigger`
 - `getNextRunTime()`
 - `skipRun()`

It is not required that container context be propagated to the threads that invoke these methods.

This is to avoid the overhead of setting up the container context when it may not be needed in these callback methods. These methods can be made contextual through the `ContextService` (see following sections), which can make any Java object contextual.

2.3.2. Contextual Objects and Tasks

Tasks are concrete implementations of the Java SE `java.util.concurrent.Callable` and `java.lang.Runnable` interfaces (see the Javadoc for `java.util.concurrent.ExecutorService`) as well as the various functional interfaces that serve as completion stage actions (see the Javadoc for `java.util.concurrent.CompletionStage`). Tasks are units of work that represent a computation or some business logic.

A contextual object is any Java object instance that has a particular application component's thread context associated with it (for example, user identity).



Contextual Objects and Tasks referred here is not the same as the Context object as defined in the Jakarta Contexts and Dependency Injection specification. See section 2.3.2.1 on *using CDI beans as tasks*.

When a task instance is submitted to a managed instance of an `ExecutorService` or a managed `CompletionStage`, the task becomes a contextual task. When the contextual task runs, the task behaves as if it were still running in the container it was submitted with.

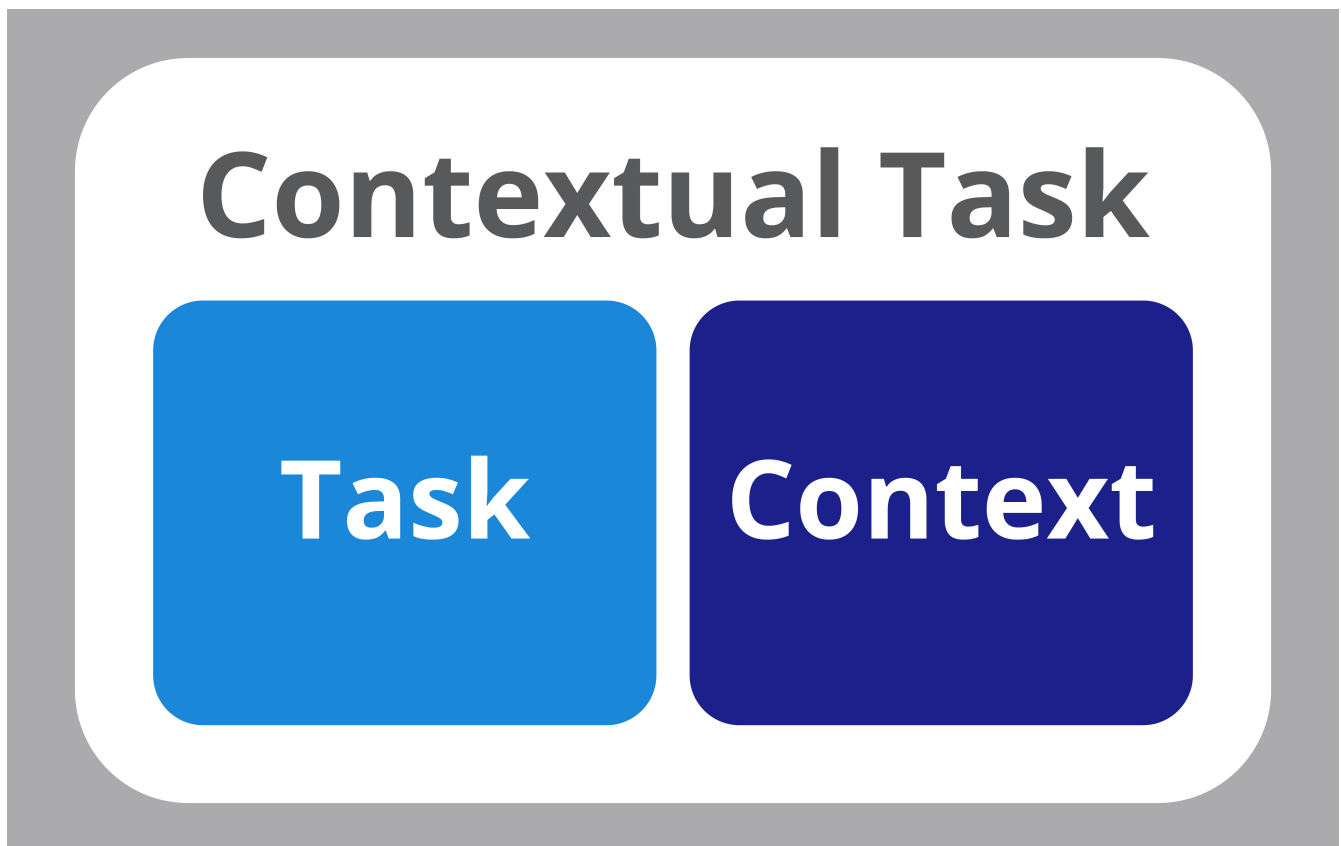


Figure 2-2 Contextual Task

2.3.2.1. Tasks and Jakarta Contexts and Dependency Injection (CDI)

CDI beans can be used as tasks. Such tasks could make use of injection if they are themselves

components or are created dynamically using various CDI APIs. However, application developers should be aware of the following when using CDI beans as tasks:

- Tasks that are submitted to a managed instance of `ExecutorService` may still be running after the lifecycle of the submitting component. Therefore, CDI beans with a scope of `@RequestScoped`, `@SessionScoped`, or `@ConversationScoped` are not recommended to use as tasks as it cannot be guaranteed that the tasks will complete before the CDI context is destroyed.
- CDI beans with a scope of `@ApplicationScoped` or `@Dependent` can be used as tasks. However, it is still possible that the task could be running beyond the lifecycle of the submitting component, such as when the component is destroyed.
- The transitive closure of CDI beans that are injected into tasks should follow the above guidelines regarding their scopes.

2.4. Usage with Jakarta Connectors

The Jakarta Connectors allows creating resource adapters that can plug into any compatible Jakarta EE application server. The Connectors specification provides a `WorkManager` interface that allows asynchronous processing for the resource adapter. It does not provide a mechanism for Jakarta EE applications to interact with an adapter's `WorkManager`.

This specification addresses the need for Jakarta EE applications to run application business logic asynchronously using a `jakarta.enterprise.concurrent.ManagedExecutorService` or `java.util.concurrent.ExecutorService` with a `jakarta.enterprise.concurrent.ManagedThreadFactory`. It is the intent that Connectors `jakarta.resource.work.WorkManager` implementations may choose to utilize or wrap the `java.util.concurrent.ExecutorService` or other functionalities within this specification when appropriate.

Resource Adapters can access each of the Managed Objects described in the following sections by looking them up in the JNDI global namespace, through the JNDI context of the accessing application (see section 11.3.2 of the Jakarta Connectors specification).

2.5. Security

This specification largely defers most security decisions to the container and Jakarta EE Product Provider as defined in the Jakarta EE Specification.

If the container supports a security context, the Jakarta EE Product Provider must propagate that security context to the thread of execution.

Application Component Providers should use the interfaces provided in this specification when interacting with threads.

Chapter 3. Managed Objects

This section introduces four programming interfaces for Jakarta EE Product Providers to implement (see EE.2.12 for a detailed definition of each of the roles described here). Instances of these interfaces must be made available to application components through containers as managed objects:

- Section 3.1, "ManagedExecutorService" –The interface for submitting asynchronous tasks from a container.
- Section 3.2, "ManagedScheduledExecutorService" – The interface for scheduling tasks to run after a given delay or execute periodically.
- Section 3.3, "ContextService" – The interface for creating contextual objects.
- Section 3.4, "ManagedThreadFactory" – The interface for creating managed threads.

3.1. ManagedExecutorService

The `jakarta.enterprise.concurrent.ManagedExecutorService` is an interface that extends the `java.util.concurrent.ExecutorService` interface. Jakarta EE Product Providers provide implementations of this interface to allow application components to run tasks asynchronously.

3.1.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE.2.12.2) use a `ManagedExecutorService` instance and associated interfaces to develop application components that utilize the concurrency functions that these interfaces provide.

The application uses the `jakarta.enterprise.concurrent.ManagedExecutorDefinition` annotation to define instances of `ManagedExecutorService` and enumerate the required qualifiers for `ManagedExecutorService` injection points that are to receive a `ManagedExecutorService` bean that is produced by the `ManagedExecutorDefinition`.

Applications can also retrieve instances using the Java Naming and Directory Interface (JNDI) Naming Context (EE.5) or through injection of resource environment references (EE.5.8.1.1).

The Application Component Provider may use resource environment references to obtain references to a `ManagedExecutorService` instance as follows:

- Assign an entry in the application component's environment to the reference using the reference type of: `jakarta.enterprise.concurrent.ManagedExecutorService`. (See EE.5.8.1.3 for information on how resource environment references are declared in the deployment descriptor.)
- Look up the managed object in the application component's environment using JNDI (EE.5.2), or through resource injection by the use of the Resource annotation (EE.5.8.1.1).

This specification recommends, but does not require, that all resource environment references be organized in the appropriate subcontext of the component's environment for the resource type. For example, all `ManagedExecutorService` references should be bound in the `java:comp/env/concurrent`

subcontext.

Components create task classes by implementing the `java.lang Runnable` or `java.util.concurrent Callable` interfaces, or any of the functional interfaces that can be supplied to a `java.util.concurrent CompletionStage`. These task classes are typically stored with the Jakarta EE application component.

Task classes can optionally implement the `jakarta.enterprise.concurrent ManagedTask` interface to provide execution properties and to register a `jakarta.enterprise.concurrent ManagedTaskListener` instance to receive lifecycle events notifications. Execution properties allow configuration and control of various aspects of the task including whether to suspend any current transaction on the thread and to provide identity information.

Task instances are submitted to a `ManagedExecutorService` instance using any of the defined `submit()`, `execute()`, `invokeAll()`, `invokeAny()`, `runAsync()`, or `supplyAsync()` methods. Task instances can also be submitted to a `CompletionStage` that is backed by a `ManagedExecutorService`. Task instances will run as an extension of the Jakarta EE container instance that submitted the task and may interact with Jakarta EE resources as defined in other sections of this specification.

It is important for Application Component Providers to identify and document the required behaviors and service-level agreements for each required `ManagedExecutorService`. The following example illustrates how the component can describe and utilize multiple executors.

3.1.1.1. Usage Example

In this example, an application component is performing two asynchronous operations from a servlet. One operation (reporter) is starting a task to generate a long running report. The other operations are short-running tasks that parallelize access to different back-end databases (builders).

Since each type of task has a completely different run profile, it makes sense to use two different `ManagedExecutorService` resource environment references. The attributes of each reference are documented using the `<description>` tag within the deployment descriptor of the application component and later mapped by the Deployer.

Reporter Task

The Reporter Task is a long-running task that communicates with a database to generate a report. The task is run asynchronously using a `ManagedExecutorService`. The client can then poll the server for the results.

Resource Environment Reference - Reporter Task

The following resource environment reference is added to the web.xml file for the web component. The description reflects the desired configuration attributes (see 3.1.4.1). Alternatively, the Resource annotation can be used in the Servlet code.



Using the description for documenting the configuration attributes of the managed object is optional. The format used here is only an example. Future revisions of Jakarta EE specifications may formalize usages such as this.

```

<resource-env-ref>
  <description>
    This executor is used for the application's reporter task.
    This executor has the following requirements:
    Context Info: Local Namespace
  </description>
  <resource-env-ref-name>
    concurrent/LongRunningTasksExecutor
  </resource-env-ref-name>
  <resource-env-ref-type>
    jakarta.enterprise.concurrent.ManagedExecutorService
  </resource-env-ref-type>
</resource-env-ref>

```

Task Definition – Reporter Task

The task itself simply uses a resource-reference to a JDBC data source, and uses a connect/use/close pattern when invoking the data source.

```

public class ReporterTask implements Runnable {
    String reportName;

    public ReporterTask(String reportName) {
        this.reportName = reportName;
    }

    public void run() {
        // Run the named report
        if("TransactionReport".equals(reportName)) {
            runTransactionReport();
        }
        else if("SummaryReport".equals(reportName)) {
            runSummaryReport();
        }
    }

    DataSource ds = ...;

    void runTransactionReport() {
        try (Connection con = ds.getConnection(); ...) {

            // Read/Write the data using our connection.
            ...
            // Commit.
            con.commit();
        }
    }
}

```

Task Submission – Reporter Task

The task is started by an HTTP client connecting to a servlet. The client specifies the report name and other parameters to run. The handle to the task (the Future) is cached so that the client can query the results of the report. The Future will contain the results once the task has completed.

```
public class AppServlet extends HttpServlet implements Servlet {

    // Cache our executor instance
    @Resource(name="/concurrent/LongRunningTasksExecutor")
    ManagedExecutorService mes;

    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
    ServletException, IOException {

        // Get the name of the report to run from the input params...
        // Assemble the header for the response.
        // Create a task instance
        ReporterTask reporterTask = new ReporterTask(reportName);

        // Submit the task to the ManagedExecutorService
        Future reportFuture = mes.submit(reporterTask);

        // Cache the future somewhere (like the client's session)
        // The client can then poll the servlet to determine
        // the status of the report.
        ...

        // Tell the user that the report has been submitted.
        ...
    }
}
```

Builder Tasks

This servlet accesses two different data sources and aggregates the results before returning the page contents to the user. Instead of accessing the data synchronously, it is instead done in parallel using two different tasks.

Resource Environment Reference – Builder Tasks

The following resource environment reference is added to the web.xml file for the web component. The description reflects the desired configuration attributes (see 3.1.4.1). Alternatively, the Resource annotation can be used in the Servlet code:



Using the description for documenting the configuration attributes of the managed object is optional. The format used here is only an example. Future revisions of Jakarta EE specifications may formalize usages such as this.

```

<resource-env-ref>
  <description>
    This executor is used for the application's builder tasks.
    This executor has the following requirements:
    Context Info: Local Namespace, Security
  </description>
  <resource-env-ref-name>
    concurrent/BuilderExecutor
  </resource-env-ref-name>
  <resource-env-ref-type>
    jakarta.enterprise.concurrent.ManagedExecutorService
  </resource-env-ref-type>
</resource-env-ref>

```

Task Definition – Builder Tasks

The task itself simply uses some mechanism such as JDBC queries to retrieve the data from the persistent store. The task implements the `jakarta.enterprise.concurrent.ManagedTask` interface and supplies an identifiable name through the `IDENTITY_NAME` property to allow system administrators to diagnose problems.

```

public class AccountTask implements Callable<AccountInfo>, ManagedTask
{
    // The ID of the request to report on demand.
    String reqID;

    String accountID;
    Map<String, String> execProps;

    public AccountTask(String reqID, String accountID) {
        this.reqID=reqID;
        this.accountID=accountID;
        execProps = new HashMap<>();
        execProps.put(ManagedTask.IDENTITY_NAME, getIdentityName());
    }

    public AccountInfo call() {

        // Retrieve account info for the account from some persistent store
        AccountInfo info = ...;
        return info;
    }

    public String getIdentityName() {
        return "AccountTask: ReqID=" + reqID + ", Acct=" + accountID;
    }

    public Map<String, String> getExecutionProperties() {
        return execProps;
    }
}

```

```

}

public ManagedTaskListener getManagedTaskListener() {
    return null;
}
}

public class InsuranceTask implements Callable<InsuranceInfo>, ManagedTask {

    // The ID of the request to report on demand.
    String reqID;

    String accountID;
    Map<String, String> execProps;

    public InsuranceTask (String reqID, String accountID) {
        this.reqID=reqID;
        this.accountID=accountID;
        execProps = new HashMap<>();

        execProps.put(ManagedTask.IDENTITY_NAME, getIdentityName());
    }

    public InsuranceInfo call() {
        // Retrieve the insurance info for the account from some persistent store
        InsuranceInfo info = ...;
        return info;
    }

    public String getIdentityName() {
        return "InsuranceTask: ReqID=" + reqID + ", Acct=" + accountID;
    }

    public Map<String, String> getExecutionProperties()
        return execProps;
    }

    public ManagedTaskListener getManagedTaskListener() {
        return null;
    }
}

```

Task Invocation – Builder Tasks

Tasks are created on demand by a request to the servlet from an HTTP client.

```

public class AppServlet extends HttpServlet implements Servlet {

    // Retrieve our executor instance.
    @Resource(name="/concurrent/BuilderExecutor")

```

```

ManagedExecutorService mes;

protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
    // Get our arguments from the request (accountNumber and
    // requestID, in this case.
    // Assemble the header for the response.
    // Create and submit the task instances

    Future<AccountInfo> acctFuture = mes.submit(new AccountTask(reqID, accountID));

    Future<InsuranceInfo> insFuture = mes.submit (new InsuranceTask(reqID, accountID)
);

    // Wait for the results.
    AccountInfo accountInfo = acctFuture.get();
    InsuranceInfo insInfo = insFuture.get();

    // Process the results
}
}

```

3.1.2. Application Assembler's Responsibilities

The Application Assembler (EE.2.12.3) is responsible for assembling the application components into a complete Jakarta EE application and providing assembly instructions that describe the dependencies to the managed objects.

3.1.3. Deployer's Responsibilities

The Deployer (EE.2.12.4) is responsible for deploying the application components into a specific operational environment. In the terms of this specification, the Deployer installs the application components and maps the dependencies defined by the Application Component Provider and Application Assembler to managed objects with the properly defined attributes. See EE.5.8.2 for details.

3.1.4. Jakarta EE Product Provider's Responsibilities

The Jakarta EE Product Provider's responsibilities are as defined in EE.5.8.3.

Jakarta EE Product Providers may include other contexts (e.g. Locale) that may be propagated to a task or a thread that invokes the callback methods in the `jakarta.enterprise.concurrent.ManagedTaskListener` interface. `ManagedExecutorService` implementations may add any additional contexts and provide the means for configuration of those contexts in any way so long as these contexts do not violate the required aspects of this specification.

The following section illustrates some possible configuration options that a Jakarta EE Product Provider may want to provide.

3.1.4.1. ManagedExecutorService Configuration Attributes

Each `ManagedExecutorService` may support one or more runtime behaviors as specified by configuration attributes. The Jakarta EE Product Provider will determine both the appropriate attributes and the means of configuring those attributes for their product.

3.1.4.2. Configuration Examples

This section and subsections illustrate some examples of how a Jakarta EE Product Provider could configure a `ManagedExecutorService` and the possible options that such a service could provide.

Providers may choose a more simplistic approach, or may choose to add more functionality, such as a higher quality-of-service, persistence, task partitioning or shared thread pools.

Each of the examples has the following attributes:

- **Name:** An arbitrary name of the service for the deployer to use as a reference.
- **JNDI name:** The arbitrary, but required, name to identify the service instance. The deployer uses this value to map the service to the component's resource environment reference.
- **Context:** A reference to a `ContextService` instance (see section 3.3). The context service can be used to define the context to propagate to the threads when running tasks. Having more than one `ContextService`, each with a different policy may be desirable for some implementations. If both `Context` and `ThreadFactory` attributes are specified, the `Context` attribute of the `ThreadFactory` configuration should be ignored.
- **ThreadFactory:** A reference to a `ManagedThreadFactory` instance (see section 3.4). The `ManagedThreadFactory` instance can create threads with different attributes (such as priority).
- **Thread Use:** If the application intends to run short vs. long-running tasks they can specify to use pooled or daemon threads.
- **Hung Task Threshold:** The amount of time in milliseconds that a task can execute before it is considered hung.
- **Pool Info:** If the executor is a thread pool, then the various thread pool attributes can be defined (this is based on the attributes for the Java `java.util.concurrent.ThreadPoolExecutor` class):
 - **Core Size:** The number of threads to keep in the pool, even if they are idle.
 - **Maximum Size:** The maximum number of threads to allow in the pool (could be unbounded).
 - **Keep Alive:** The time to allow threads to remain idle when the number of threads is greater than the core size.
 - **Work Queue Capacity:** The number of tasks that can be stored in the input bounded buffer (could be unbounded).
- **Reject Policy:** The policy to use when a task is to be rejected by the executor. In this example, two policies are available:
 - **Abort:** Throw an exception when rejected.
 - **Retry and Abort:** Automatically resubmit to another instance and abort if it fails.

Typical Thread Pool

The Typical Thread Pool illustrates a common configuration for an application server with few applications. Each application expects to run a small number of short-duration tasks in the local process.

ManagedExecutorService	
Name:	Typical Thread Pool
JNDI Name:	concurrent/execsvc/Shared
Context:	concurrent/ctx/AllContexts
Thread Factory:	concurrent/tf/normal
Hung Task Threshold	60000 ms
Pool Info:	Core Size: 5 Max Size: 25 Keep Alive: 5000 ms Work Queue: 15 Capacity:
Reject Policy	☒ Abort ☐ Retry and Abort

##Table : Typical Thread Pool Configuration Example

Thread Pool for Long-Running Tasks

This executor describes a configuration in which the executor is used to run a few long-running tasks in the local process. In this example the task can run up to 24 hours before it is considered hung.

ManagedExecutorService	
Name:	Long-Running Tasks Thread Pool
JNDI Name:	concurrent/execsvc/LongRunning
Context:	concurrent/ctx/AllContexts
Thread Factory:	concurrent/tf/longRunningThreadsFactory
Hung Task Threshold	24 hours

Pool Info:	Core Size: 0 Max Size: 5 Keep Alive: 1000 ms Work Queue: 5 Capacity:
Reject Policy	☐ Abort ☐ Retry and Abort

##Table : Long-Running Tasks Thread Pool Configuration Example

OLTP Thread Pool

The OLTP (On-Line Transaction Processing) Thread Pool executor uses a thread pool with many more threads and a low hung-task threshold. It also uses a thread factory that creates threads with a slightly higher priority and a ContextService with a limited amount of context information.

ManagedExecutorService	
Name:	Shared OLTP Thread Pool
JNDI Name:	concurrent/execsvc/OLTPShared
Context:	concurrent/ctx/OLTPContexts
Thread Factory:	concurrent/tf/oltp
Hung Task Threshold	20000 ms
Pool Info:	Core Size: 100 Max Size: 250 Keep Alive: 10000 ms Work Queue: 100 Capacity:
Reject Policy	☐ Abort ☐ Retry and Abort

##Table : OLTP Thread Pool Configuration Example

3.1.4.3. Default ManagedExecutorService

The Jakarta EE Product Provider must provide a preconfigured, default ManagedExecutorService for use by application components under the JNDI name `java:comp/DefaultManagedExecutorService`. The types of contexts to be propagated by this default `ManagedExecutorService` from a contextualizing application component must include naming context, classloader, and security

information.

The Jakarta EE Product Provider must inject the default `ManagedExecutorService` into injection points of `ManagedExecutorService` that do not have any qualifiers except for where the application provides the producer, in which case the application's producer takes precedence.

3.1.5. System Administrator's Responsibilities

The System Administrator (EE.2.12.5) is responsible for monitoring and overseeing the runtime environment. In the scope of this specification, these duties may include:

- monitoring for hung tasks
- monitoring resource usage (for example, threads and memory)

3.1.6. Lifecycle

The lifecycle of `ManagedExecutorService` instances are centrally managed by the application server and cannot be changed by an application.

A `ManagedExecutorService` instance is intended to be used by multiple components and applications. When the executor runs a task, the context of the thread is changed to match the component instance that submitted the task. The context is then restored when the task is complete.

In Figure 3-1, a single `ManagedExecutorService` instance is used to run tasks (in blue) from multiple application components (each denoted in a different color). Each task, when submitted to the `ManagedExecutorService` automatically retains the context of the submitting component and it becomes a Contextual Task. When the `ManagedExecutorService` runs the task, the task would be run in the context of the submitting component (as noted by different colored boxes in the figure).

APPLICATION SERVER PROCESS

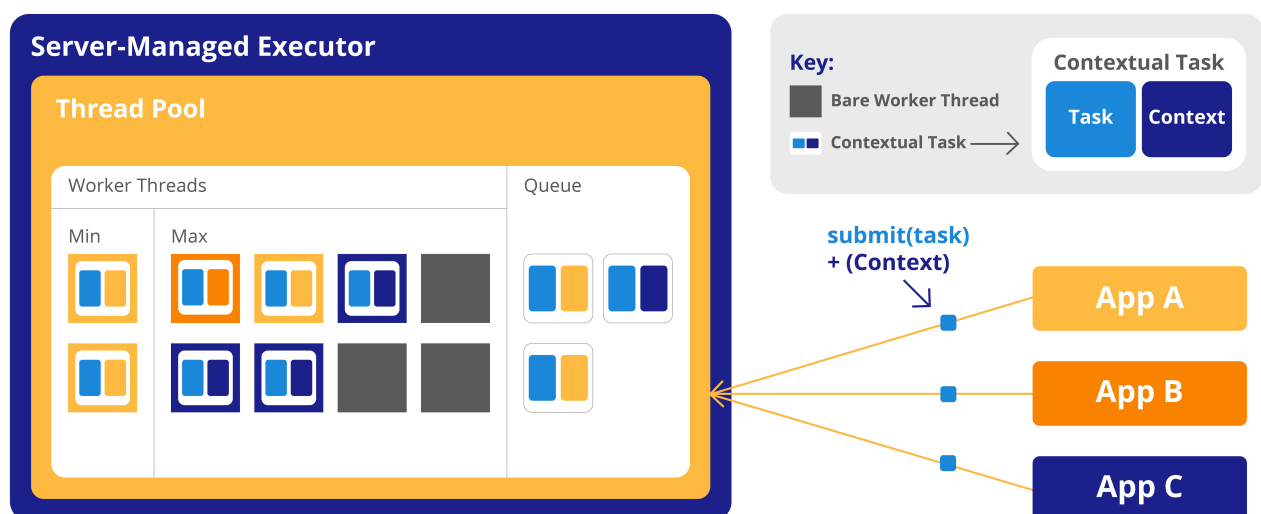


Figure 3-1 Managed Thread Pool Executor Component Relationship

`ManagedExecutorService` instances may be terminated or suspended by the application server when applications or components are stopped or the application server itself is shutting down.

3.1.6.1. Jakarta EE Product Provider Requirements

This subsection describes additional requirements for `ManagedExecutorService` providers.

1. All tasks, when executed from the `ManagedExecutorService`, will run with the Jakarta EE component identity of the component that submitted the task.
2. The lifecycle of a `ManagedExecutorService` is managed by an application server. All lifecycle operations on the `ManagedExecutorService` interface will throw a `java.lang.IllegalStateException` exception. This includes the following methods that are defined in the `java.util.concurrent.ExecutorService` interface: `awaitTermination()`, `isShutdown()`, `isTerminated()`, `shutdown()`, and `shutdownNow()`.
3. No task submitted to an executor can run if task's component is not started.

When a `ManagedExecutorService` instance is being shutdown by the Jakarta EE Product Provider:

1. All attempts to submit new tasks are rejected.
2. All submitted tasks are cancelled if not running.
3. All running task threads are interrupted.
4. All registered `ManagedTaskListeners` are invoked.

3.1.7. Quality of Service

`ManagedExecutorService` implementations must support the at-most-once quality of service. The at-most-once quality of service guarantees that a task will run at most one time. This quality of service is the most efficient method to run tasks. Tasks submitted to an executor with this quality of service are transient in nature, are not persisted, and do not survive process restarts.

Other qualities of service are allowed, but are not addressed in this specification.

3.1.8. Transaction Management

`ManagedExecutorService` implementations must support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions specification. User-managed transactions allow components to manually control global transaction demarcation boundaries. Task implementations may optionally begin, commit, and roll-back a transaction. See EE.4 for details on transaction management in Jakarta EE.

Task instances are run outside of the scope of the transaction of the submitting thread. Any transaction active in the executing thread will be suspended.

3.1.8.1. Jakarta EE Product Provider Requirements

This subsection describes the transaction management requirements of a `ManagedExecutorService` implementation.

1. The `jakarta.transaction.UserTransaction` interface must be made available in the local JNDI namespace as environment entry: `java:comp/UserTransaction` (EE.5.10 and EE.4.2.1.1)
2. All resource managers must enlist with a `UserTransaction` instance when a transaction is active

using the `begin()` method.

3. The executor is responsible for coordinating commits and rollbacks when the transaction ends using `commit()` and `rollback()` methods.
4. A task must have the same ability to use transactions as the component submitting the tasks. For example, tasks are allowed to call transactional enterprise beans, and managed beans that use the `@Transactional` interceptor as defined in the Jakarta Transactions specification.

3.1.8.2. Application Component Provider's Requirements

This subsection describes the transaction management requirements of each task provider's implementation.

1. A task instance that starts a transaction must complete the transaction before starting a new transaction.
2. The task provider uses the `jakarta.transaction.UserTransaction` interface to demarcate transactions.
3. Transactions are demarcated using the `begin()`, `commit()` and `rollback()` methods of the `UserTransaction` interface.
4. While an instance is in an active transaction, resource-specific transaction demarcation APIs must not be used (e.g., if a `java.sql.Connection` is enlisted in the transaction instance, the `Connection.commit()` and `Connection.rollback()` methods must not be used).
5. The task instance must complete the transaction before the task method ends.

UserTransaction Usage Example

The following example illustrates how a task can interact with two XA-capable resources in a single transaction:

```
public class TranTask implements Runnable {

    UserTransaction ut = ...;

    public void run() {

        // Start a transaction
        ut.begin();

        // Invoke an Jakarta Enterprise Bean
        ...

        //
        Update a database using an XA capable JDBC DataSource
        ...

        // Commit the transaction
        ut.commit();
    }
}
```

3.2. ManagedScheduledExecutorService

The `jakarta.enterprise.concurrent.ManagedScheduledExecutorService` is an interface that extends the `java.util.concurrent.ScheduledExecutorService` and `jakarta.enterprise.concurrent.ManagedExecutorService` interfaces. Jakarta EE Product Providers provide implementations of this interface to allow applications to run tasks at specified and periodic times.

The `ManagedScheduledExecutorService` offers the same managed semantics as the `ManagedExecutorService` and includes the delay and periodic task running capabilities that the `ScheduledExecutorService` interface provides with the addition of `Trigger` and `ManagedTaskListener`.

3.2.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE2.12.2) use a `ManagedScheduledExecutorService` instance and associated interfaces to develop application components that utilize the concurrency functions that these interfaces provide.

The application uses the `jakarta.enterprise.concurrent.ManagedScheduledExecutorDefinition` annotation to define instances of `ManagedScheduledExecutorService` and enumerate the required qualifiers for `ManagedScheduledExecutorService` injection points that are to receive a `ManagedScheduledExecutorService` bean that is produced by the `ManagedScheduledExecutorDefinition`.

Applications can also retrieve instances using the Java Naming and Directory Interface (JNDI) Naming Context (EE.5.2) or through injection of resource environment references (EE.5.8.1.1).

The Application Component Provider may use resource environment references to obtain references to a `ManagedScheduledExecutorService` instance as follows:

- Assign an entry in the application component's environment to the reference using the reference type of: `jakarta.enterprise.concurrent.ManagedScheduledExecutorService`. (See EE.5.8.1.2 for information on how resource environment references are declared in the deployment descriptor.)
- Look up the managed object in the application component's environment using JNDI (EE.5.2), or through resource injection by the use of the `@Resource` annotation (EE.5.8.1.1).

This specification recommends, but does not require, that all resource environment references be organized in the appropriate subcontext of the component's environment for the resource type. For example, all `ManagedScheduledExecutorService` references should be declared in the `java:comp/env/concurrent` subcontext.

Components create task classes by implementing the `java.lang.Runnable` or `java.util.concurrent.Callable` interfaces. These task classes are typically stored with the Jakarta EE application component.

Task instances are submitted to a `ManagedScheduledExecutorService` instance using any of the

defined `submit()`, `execute()`, `invokeAll()`, `invokeAny()`, `runAsync()`, `supplyAsync()`, `schedule()`, `scheduleAtFixedRate()`, or `scheduleWithFixedDelay()` methods. Task instances can also be submitted to a `CompletionStage` that is backed by a `ManagedScheduledExecutorService`. Task instances will run as an extension of the Jakarta EE container instance that submitted the task and may interact with Jakarta EE resources as defined in other sections of this specification.

Task classes can optionally implement the `jakarta.enterprise.concurrent.ManagedTask` interface to provide execution properties and to register a `jakarta.enterprise.concurrent.ManagedTaskListener` instance to receive lifecycle events notifications. Execution properties allow configuration and control of various aspects of the task including whether to suspend any current transaction on the thread and to provide identity information.

It is important for Application Component Providers to identify and document the required behaviors and service-level agreements for each required `ManagedScheduledExecutorService`. The following example illustrates how the component can describe and utilize a `ManagedScheduledExecutorService`.

3.2.1.1. Usage Example

In this example, an application component wants to use a timer to periodically write in-memory events to a database log.

The attributes of the `ManagedScheduledExecutorService` reference are documented using the `<description>` tag within the deployment descriptor of the application component and later mapped by the Deployer.

Logger Timer Task

The Logger Timer Task is a short-running, periodic task that has the same lifecycle as the servlet. It periodically wakes up and dumps a queue's contents to a database log. Its lifecycle is controlled using a `jakarta.servlet.ServletContextListener`.

Resource Environment Reference

The following resource environment reference is added to the `web.xml` file for the web component. The description reflects the desired configuration attributes (see 3.2.4.1). Alternatively, the Resource annotation can be used in the Servlet code.



Using the description for documenting the configuration attributes of the managed object is optional. The format used here is only an example. Future revisions of Jakarta EE specifications may formalize usages such as this.

```
<resource-env-ref>
  <description>
    This executor is used for the application's logger task.
    This executor has the following requirements:
    Context Info: Local Namespace
  </description>
</resource-env-ref-name>
```

```

    concurrent/ScheduledLoggerExecutor
</resource-env-ref-name>
<resource-env-ref-type>
    jakarta.enterprise.concurrent.ManagedScheduledExecutorService
</resource-env-ref-type>
</resource-env-ref>

```

Task Definition

The task itself simply uses a resource-reference to a JDBC data source, and uses a connect/use/close pattern when invoking the data source.

```

public class LoggerTimer implements Runnable {
    DataSource ds = ...;

    public void run() {
        logEvents(getData(), ds);
    }

    void logEvents(Collection data, DataSource ds) {

        // Iterate through the data and log each row.
        for (...) {
            try (Connection con = ds.getConnection(); ...) {

                // Write the data using our connection.
                ...

                // Commit.
                con.commit();
            }
        }
    }
}

```

Task Submission

The task is started and stopped by a `jakarta.servlet.ServletContextListener`.

```

public class CtxListener implements ServletContextListener {

    Future loggerHandle = null;

    @Resource(name="concurrent/ScheduledLoggerExecutor")
    ManagedScheduledExecutorService mes;

    public void contextInitialized(ServletContextEvent scEvent) {
        LoggerTimer logger = new LoggerTimer();
        loggerHandle = mes.scheduleAtFixedRate(logger,

```

```

        5, // initial delay
        60, // period
        TimeUnit.SECONDS);
    }

    public void contextDestroyed(ServletContextEvent scEvent) {

        // Cancel and interrupt our logger task
        if(loggerHandle!=null) {
            loggerHandle.cancel(true);
        }
    }
}

```

3.2.2. Application Assembler’s Responsibilities

The Application Assembler (EE.2.12.3) is responsible for assembling the application components into a complete Jakarta EE Application and providing assembly instructions that describe the dependencies to the managed objects.

3.2.3. Deployer’s Responsibilities

The Deployer (EE.2.12.4) is responsible for deploying the application components into a specific operational environment. In the terms of this specification, the Deployer installs the application components and maps the dependencies defined by the Application Component Provider and Application Assembler to managed objects with the properly defined attributes. See EE.5.8.2 for details.

3.2.4. Jakarta EE Product Provider’s Responsibilities

The Jakarta EE Product Provider’s responsibilities are as defined in EE.5.8.3.

Jakarta EE Product Providers may include other contexts that may be propagated to a task or `jakarta.enterprise.concurrent.ManagedTaskListener` thread (e.g. `Locale`). `ManagedScheduledExecutorService` implementations may add any additional contexts and provide the means for configuration of those contexts in any way so long as these contexts do not violate the required aspects of this specification.

The following section illustrates some possible configuration options that a Jakarta EE Product Provider may want to provide.

3.2.4.1. ManagedScheduledExecutorService Configuration Attributes

Each `ManagedScheduledExecutorService` may support one or more runtime behaviors as specified by configuration attributes. The Jakarta EE Product Provider will determine both the appropriate attributes and the means of configuring those attributes for their product.

3.2.4.2. Configuration Examples

This section and subsections illustrate some examples of how a Jakarta EE Product Provider could configure a `ManagedScheduledExecutorService` and the possible options that such a service could provide.

Providers may choose a more simplistic approach, or may choose to add more functionality, such as a higher quality-of-service or persistence.

Each of the examples has the following attributes:

- **Name:** An arbitrary name of the service for the deployer to use as a reference.
- **JNDI name:** The arbitrary, but required, name to identify the service instance. The deployer uses this value to map the service to the component's resource environment reference.
- **Context:** A reference to a `ContextService` instance (see section 3.3). The context service can be used to define the context to propagate to the threads when running tasks. Having multiple `ContextService` instances, each with a different policy, may be desirable for some implementations. If both `Context` and `ThreadFactory` attributes are specified, the `Context` attribute of the `ThreadFactory` configuration should be ignored.
- **ThreadFactory:** A reference to a `ManagedThreadFactory` instance (see section 3.4). The managed `ThreadFactory` instance can create threads with different attributes (such as priority).
- **Thread Use:** If the application intends to run short vs. long-running tasks they can specify to use pooled or daemon threads.
- **Hung Task Threshold:** The amount of time in milliseconds that a task can execute before it is considered hung.
- **Pool Info:** If the executor is a thread pool, then the various thread pool attributes can be defined (this is based on the attributes for the Java `java.util.concurrent.ThreadPoolExecutor` class):
 - **Core Size:** The number of threads to keep in the pool, even if they are idle.
 - **Maximum Size:** The maximum number of threads to allow in the pool (could be unbounded).
 - **Keep Alive:** The time to allow threads to remain idle when the number of threads is greater than the core size.
- **Reject Policy:** The policy to use when a task is to be rejected by the executor. In this example, two policies are available:
 - **Abort:** Throw an exception when rejected.
 - **Retry and Abort:** Automatically resubmit to another instance and abort if it fails.

Typical Timer



This example describes a typical configuration for a `ManagedScheduledExecutorService` that uses a bounded thread pool. Only 10 timers can run simultaneously and are considered hung if they have run more than 5 seconds. An executor such as this can be shared between applications and is

designed to run very short-duration tasks, for example, marking a transaction to roll back after a timeout.

ManagedScheduledExecutorService	
Name:	Typical Timer
JNDI Name:	concurrent/excsvc/Timer
Context:	concurrent/ctx/AllContexts
Thread Factory:	concurrent/tf/normal
Thread Use:	☒ Pooled ☐ Daemon
Hung Task Threshold	5000 ms
Pool Info:	Core Size: 2 Max Size: 10 Keep Alive: 3000 ms
Reject Policy	☐ Abort ☐ Retry and Abort

##Table : Typical Timer Configuration Example

3.2.4.3. Default ManagedScheduledExecutorService

The Jakarta EE Product Provider must provide a preconfigured, default **ManagedScheduledExecutorService** for use by application components under the JNDI name **java:comp/DefaultManagedScheduledExecutorService**. The types of contexts to be propagated by this default **ManagedScheduledExecutorService** from a contextualizing application component must include naming context, class loader, and security information.

The Jakarta EE Product Provider must inject the default **ManagedScheduledExecutorService** into injection points of **ManagedScheduledExecutorService** that do not have any qualifiers except for where the application provides the producer, in which case the application's producer takes precedence.

3.2.5. System Administrator's Responsibilities

The System Administrator (EE.2.12.5) is responsible for monitoring and overseeing the runtime environment. In the scope of this specification, these duties may include:

- Monitoring for hung tasks.
- Monitoring resource usage (for example, threads and memory).

3.2.6. Lifecycle

The lifecycle of **ManagedScheduledExecutorService** instances are centrally managed by the application

server and cannot be changed by an application.

A `ManagedScheduledExecutorService` instance can be used by multiple components and applications. When the executor runs a task, the context of the thread is changed to match the component instance that submitted the task. The context is then restored when the task is complete. See Figure 3-1 Managed Thread Pool Executor Component Relationship.

`ManagedScheduledExecutorService` instances may be terminated or suspended by the application server when applications or components are stopped or the application server itself is shutting down.

3.2.6.1. Jakarta EE Product Provider Requirements

This subsection describes requirements for `ManagedScheduledExecutorService` providers.

1. All tasks, when executed from the `ManagedScheduledExecutorService`, will run with the context of the application component that submitted the task.
2. The lifecycle of a `ManagedScheduledExecutorService` is managed by an application server. All lifecycle operations on the `ManagedScheduledExecutorService` interface will throw a `java.lang.IllegalStateException` exception. This includes the following methods that are defined in the `java.util.concurrent.ExecutorService` interface: `awaitTermination()`, `isShutdown()`, `isTerminated()`, `shutdown()`, and `shutdownNow()`.
3. All tasks submitted to an executor must not run if task's component is not started.

When a `ManagedScheduledExecutorService` instance is being shutdown by the Jakarta EE Product Provider:

1. All attempts to submit new tasks are rejected.
2. All submitted tasks are cancelled if not running.
3. All running task threads are interrupted.
4. All registered `ManagedTaskListeners` are invoked.

3.2.7. Quality of Service

`ManagedScheduledExecutorService` implementations must support the at-most-once quality of service. The at-most-once quality of service guarantees that a task will run at most, one time. This quality of service is the most efficient method to run tasks. Tasks submitted to an executor with this quality of service are transient in nature, are not persisted, and do not survive process restarts.

Other qualities of service are allowed, but are not addressed in this specification.

3.2.8. Transaction Management

`ManagedScheduledExecutorService` implementations must support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions specification. User-managed transactions allow components to manually control global transaction demarcation boundaries. Task implementations may optionally begin, commit, and roll-back a transaction. See EE.4 for details on transaction management in Jakarta EE.

Task instances are run outside of the scope of the transaction of the submitting thread. Any transaction active in the executing thread will be suspended.

3.2.8.1. Jakarta EE Product Provider Requirements

This subsection describes the transaction management requirements of a `ManagedScheduledExecutorService` implementation.

1. The `jakarta.transaction.UserTransaction` interface must be made available in the local JNDI namespace as environment entry: `java:comp/UserTransaction` (J2EE.5.7 and J2EE.4.2.1.1)
2. All resource managers must enlist with a `UserTransaction` instance when a transaction is active using the `begin()` method.
3. The executor is responsible for coordinating commits and rollbacks when the transaction ends using `commit()` and `rollback()` methods.
4. A task must have the same ability to use transactions as the component submitting the tasks. For example, tasks are allowed to call transactional enterprise beans, and managed beans that use the `@Transactional` interceptor as defined in the Jakarta Transactions specification.

3.2.8.2. Application Component Provider's Requirements

This subsection describes the transaction management requirements of each task provider's implementation.

1. A task instance that starts a transaction must complete the transaction before starting a new transaction.
2. The task provider uses the `jakarta.transaction.UserTransaction` interface to demarcate transactions.
3. Transactions are demarcated using the `begin()`, `commit()` and `rollback()` methods of the `UserTransaction` interface.
4. While an instance is in an active transaction, resource-specific transaction demarcation APIs must not be used (e.g., if a `java.sql.Connection` is enlisted in the transaction instance, the `Connection.commit()` and `Connection.rollback()` methods must not be used).
5. The task instance must complete the transaction before the task method ends.

See the example titled *UserTransaction Usage Example* under section 3.1.8.2 for an example on how to use a `UserTransaction` within a task.

3.3. ContextService

The `jakarta.enterprise.concurrent.ContextService` allows applications to create contextual objects without using a managed executor. The `ContextService` uses the dynamic proxy capabilities found in the `java.lang.reflect` package or creates proxy instances in a non-dynamic manner to associate the application component container context with an object instance. The object becomes a contextual object (see section 2.3.2) and whenever a method on the contextual object is invoked, the method executes with the thread context of the associated application component instance.

Contextual objects allow application components to develop a wide variety of applications and services that are not normally possible in the Jakarta EE platform, such as workflow systems. When used in conjunction with a `ManagedThreadFactory`, customized Java SE platform `ExecutorService` implementations can be used.

The `ContextService` also allows non-Jakarta EE service callbacks (such as Jakarta Messaging `MessageListeners` and JMX `NotificationListeners`) to run in the context of the listener registrant instead of the implementation provider's undefined thread context.).

3.3.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE2.12.2) use a `ContextService` instance to create contextual object proxies.

The application uses the `jakarta.enterprise.concurrent.ContextServiceDefinition` annotation to define instances of `ContextService` and enumerate the required qualifiers for `ContextService` injection points that are to receive a `ContextService` bean that is produced by the `ContextServiceDefinition`.

Applications can also retrieve instances using the Java Naming and Directory Interface (JNDI) Naming Context (EE.5) or through injection of resource environment references (EE.5.8.1.1).

The Application Component Provider may use resource environment references to obtain references to a `ContextService` instance as follows:

- Assign an entry in the application component's environment to the reference using the reference type of: `jakarta.enterprise.concurrent.ContextService`. (See EE.5.8.1.2 for information on how resource environment references are declared in the deployment descriptor.)
- Look up the managed object in the application component's environment using JNDI (EE.5.2), or through resource injection by the use of the `@Resource` annotation (EE.5.8.1.1).

This specification recommends, but does not require, that all resource environment references be organized in the appropriate subcontext of the component's environment for the resource type. For example, all `ContextService` references should be declared in the `java:comp/env/concurrent` subcontext.

- Contextual object proxy instances are created with a `ContextService` instance using the `createContextualProxy()` or `contextual*()` methods. Contextual object proxies will run as an extension of the application component instance that created the proxy and may interact with Jakarta EE container resources as defined in other sections of this specification.
- Specialized contextual proxies for unmanaged `CompletionStage` and `CompletableFuture` instances are created with the `withContextCapture()` methods, enabling context propagation to all dependent stages.

It is important for Application Component Providers to identify and document the required behaviors and service-level agreements for each required `ContextService`. The following example illustrates how the component can describe and utilize a `ContextService`.

3.3.1.1. Usage Example

This section provides an example that shows how a custom `ExecutorService` can be utilized within an application component.

Custom ExecutorService

This example demonstrates how a singleton Java SE `ExecutorService` implementation (such as the `java.util.concurrent.ThreadPoolExecutor`) can be used from a Jakarta Enterprise Bean. In this example, the reference `ThreadPoolExecutor` implementation is used instead of the implementation supplied with the Jakarta EE Product Provider.

A custom `ExecutorService` can be created like any Java object. For applications to use an object, it can be accessed using a singleton or using a Connectors resource adapter. In this example, we use a singleton session bean.

Since the `ExecutorService` is a singleton session bean, it can be accessed by several Jakarta Enterprise Beans or Servlet instances. The `ExecutorService` uses threads created from a `ManagedThreadFactory` (see section 3.4) provided by the Jakarta EE Product Provider. The `ContextService` is used to guarantee that the task, when it runs on one of the worker threads in the pool, will have the correct component context available to it.

ExecutorService Singleton

Create a singleton session bean `ExecutorAccessor` with a getter for the `ExecutorService`. The `ExecutorAccessor` should be included with the enterprise bean module or other jar that is in the scope of the application component.

```
@Singleton
public class ExecutorAccessor {

    private ExecutorService threadPoolExecutor = null;

    @Resource(name="concurrent/ThreadFactory")
    ManagedThreadFactory threadFactory;

    @PostConstruct
    public void postConstruct() {
        threadPoolExecutor = new ThreadPoolExecutor( 5, 10, 5, TimeUnit.SECONDS, new
        ArrayBlockingQueue<Runnable>(10), threadFactory);
    }

    public ExecutorService getThreadPool() {
        return threadPoolExecutor;
    }
}
```

CreditReport Task

The `CreditReport` task retrieves a credit report from a given credit agency for a given tax

identification number. Multiple tasks are invoked in parallel by an Enterprise Bean business method.

Resource Environment References

This example refers to a `ContextService` and a `ManagedThreadFactory`.



Using the description for documenting the configuration attributes of the managed object is optional. The format used here is only an example. Future revisions of Jakarta EE specifications may formalize usages such as this.

```
<resource-env-ref>
  <description>
    This ThreadFactory is used for the singleton ThreadPoolExecutor.
    Priority: Normal
    Context Info: NA
  </description>

  <resource-env-ref-name>
    concurrent/ThreadFactory
  </resource-env-ref-name>

  <resource-env-ref-type>
    jakarta.enterprise.concurrent.ManagedThreadFactory
  </resource-env-ref-type>
</resource-env-ref>

<resource-env-ref>
  <description>
    This ContextService is used in conjunction with the custom
    ThreadPoolExecutor that the credit report component is using.
    This ContextService has the following requirements:
    Context Info: Local namespace, security
  </description>

  <resource-env-ref-name>
    concurrent/AllContexts
  </resource-env-ref-name>

  <resource-env-ref-type>
    jakarta.enterprise.concurrent.ContextService
  </resource-env-ref-type>
</resource-env-ref>
```

Task Definition

This task logs the request in a database, which requires the local namespace in order to locate the correct `DataSource`. It also utilizes the Java Authentication and Authorization API (JAAS) to retrieve the user's identity from the current thread in order to audit access to the credit report.

```

public class CreditScoreTask implements Callable<Long> {
    private long taxID;
    private int agency;

    public CreditScoreTask(long taxID, int agency) {
        this.taxID = taxID;
        this.agency = agency;
    }

    public Long call() {
        // Log the request in a database using the identity of the user.
        // Use the local namespace to locate the datasource
        Subject currentSubject = Subject.getSubject(AccessController.getContext());
        logCreditAccess(currentSubject, taxID, agency);

        // Use Web Services to retrieve the credit score from the
        // specified agency.
        return getCreditScore(taxID, agency);
    }

    ...
}

```

Task Invocation

The `LoanCheckerBean` is a stateless session bean that has one method that is used to retrieve the credit scores for one tax ID from three different agencies. It uses three threads to accomplish this, including the enterprise bean thread.

While the enterprise bean thread is retrieving one credit score, two other threads are retrieving the other two scores.

```

class LoanCheckerBean {
    @Resource(name="concurrent/AllContexts")
    ContextService ctxSvc;

    @EJB private ExecutorAccessor executorAccessor;

    public long[] getCreditScores(long taxID) {
        // Retrieve our singleton threadpool, but wrap it in
        // a ExecutorCompletionService
        ExecutorCompletionService<Long> threadPool = new ExecutorCompletionService<Long>
(executorAccessor.getThreadPool());

        // Use this thread to retrieve one credit score, and
        // use two other threads to process the other two scores.
        // Since we are using a custom executor and
        // because our tasks depend upon the context in which this
        // method is running, we use a contextual task.
    }
}

```

```

CreditScoreTask agency1 = new CreditScoreTask(taxID, 1);

Callable<Long> agency2 = ctxSvc.createContextualProxy( new CreditScoreTask(taxID,
2), Callable.class));

Callable<Long> agency3 = ctxSvc.createContextualProxy ( new CreditScoreTask(taxID,
3), Callable.class));

threadPool.submit(agency2);
threadPool.submit(agency3);

long[] scores = {0,0,0};
try {
    // Retrieve one credit score on this thread.
    scores[0] = agency1.call();

    // Retrieve the other two credit scores
    scores[1] = threadPool.take().get();
    scores[2] = threadPool.take().get();
} catch (InterruptedException e) {
    // The app may be shutting down.
} catch (ExecutionException e) {
    // There was an error retrieving one of the asynch scores.
}
return scores;
}
}

```

3.3.2. Application Assembler's Responsibilities

The Application Assembler (EE.2.12.3) is responsible for assembling the application components into a complete Jakarta EE Application and providing assembly instructions that describe the dependencies to the managed objects.

3.3.3. Deployer's Responsibilities

The Deployer (EE.2.12.4) is responsible for deploying the application components into a specific operational environment. In the terms of this specification, the Deployer installs the application components and maps the dependencies defined by the Application Component Provider and Application Assembler to managed objects with the properly defined attributes. See EE.5.8.2 for details.

All objects created by a `ContextService` instance are required to propagate Jakarta EE container context information (see section 2.3) to the methods invoked on the proxied object.

3.3.4. Jakarta EE Product Provider's Responsibilities

The Jakarta EE Product Provider's responsibilities are as defined in EE.5.8.3 and must provide an implementation of any behaviors defined in the following:

- All invocation handlers for the contextual proxy implementation must implement `java.io.Serializable`.
- All invocations to any of the proxied interface methods will fail with a `java.lang.IllegalStateException` exception if the application component is not started or deployed.

Jakarta EE Product Providers may add any additional container contexts to the managed `ContextService` and provide the means for configuration of those contexts in any way so long as these contexts do not violate the required aspects of this specification.

The following section illustrates some possible configuration options that a Jakarta EE Product Provider may want to provide.

3.3.4.1. ContextService Configuration Attributes

Each `ContextService` may support one or more runtime behaviors as specified by configuration attributes. The Jakarta EE Product Provider will determine both the appropriate attributes and the means of configuring those attributes for their product.

3.3.4.2. Configuration Examples

This section and subsections illustrate some examples how a Jakarta EE Product Provider could configure a `ContextService` and the possible options that such a service could provide.

The `ContextService` can be used directly by application components by using resource environment references or providers may choose to use the context information supplied as default context propagation policies for a `ManagedExecutorService`, `ManagedScheduledExecutorService` or `ManagedThreadFactory`. The configuration examples covered in sections 3.1.4.2, 3.2.4.2 and 3.4.4.2 refer to one of the `ContextService` configuration examples that follow.

Each of the examples has the following attributes:

- **Name:** An arbitrary name of the service for the deployer to use as a reference.
- **JNDI name:** The arbitrary, but required, name to identify the service instance. The deployer uses this value to map the service to the component's resource environment reference.
- **Context info:** The context information to be propagated.
 - **Security:** If enabled, propagate the container security principal.
 - **Locale:** If enabled, the locale from the container thread is propagated.
 - **Custom:** If enabled, custom, thread-local data is propagated.

All Contexts

ContextService	
Name:	All Contexts
JNDI Name:	Concurrent/cs/AllContexts

Context Info:	☒ Security ☒ Locale ☒ Custom
---------------	--

##Table : All Contexts Configuration Example

OLTP Contexts

ContextService	
Name:	OLTP Contexts
JNDI Name:	Concurrent/cs/OLTPContexts
Context Info:	☒ Security ☐ Locale ☒ Custom

##Table : OLTP Contexts Configuration Example

No Contexts

ContextService	
Name:	No Contexts
JNDI Name:	Concurrent/cs/NoContexts
Context Info:	Security Locale Custom

##Table : No Contexts Configuration Example

3.3.4.3. Default ContextService

The Jakarta EE Product Provider must provide a preconfigured, default `ContextService` for use by application components under the JNDI name `java:comp/DefaultContextService`. The types of contexts to be propagated by this default `ContextService` from a contextualizing application component must include naming context, class loader, and security information.

The Jakarta EE Product Provider must inject the default `ContextService` into injection points of `ContextService` that do not have any qualifiers except for where the application provides the producer, in which case the application's producer takes precedence.

3.3.5. Transaction Management

Contextual dynamic proxies support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions

specification. By default, proxy methods suspend any transactional context on the thread and allow components to manually control global transaction demarcation boundaries. Context objects may optionally begin, commit, and roll back a transaction. See EE.4 for details on transaction management in Jakarta EE.

By using an execution property when creating the contextual proxy object, application components can choose to not suspend the transactional context on the thread, and any resources used by the task will be enlisted to that transaction. Refer to the Javadoc for the `jakarta.enterprise.concurrent.ContextService` interface for details and examples.

3.3.5.1. Jakarta EE Product Provider Requirements

This subsection describes the transaction management requirements of a `ContextService` implementation when transaction management is enabled (this is the default behavior).

1. The `jakarta.transaction.UserTransaction` interface must be made available in the local JNDI namespace as environment entry: `java:comp/UserTransaction` (EE.5.10 and EE.4.2.1.1)
2. All resource managers must enlist with a `UserTransaction` instance when a transaction is active using the `begin()` method.
3. The executor is responsible for coordinating commits and rollbacks when the transaction ends using `commit()` and `rollback()` methods.
4. A task must have the same ability to use transactions as the component submitting the tasks. For example, tasks are allowed to call transactional enterprise beans, and managed beans that use the `@Transactional` interceptor as defined in the Jakarta Transactions specification.

3.3.5.2. Application Component Provider's Requirements

This subsection describes the transaction management requirements of each task provider's implementation when transaction management is enabled (this is the default behavior).

1. A task instance that starts a transaction must complete the transaction before starting a new transaction.
2. The task provider uses the `jakarta.transaction.UserTransaction` interface to demarcate transactions.
3. Transactions are demarcated using the `begin()`, `commit()` and `rollback()` methods of the `UserTransaction` interface.
4. While an instance is in an active transaction, resource-specific transaction demarcation APIs must not be used (e.g. if a `java.sql.Connection` is enlisted in the transaction instance, the `Connection.commit()` and `Connection.rollback()` methods must not be used).
5. The task instance must complete the transaction before the task method ends.

See the example titled *UserTransaction Usage Example* under section 3.1.8.2 for an example of using a `UserTransaction` within a task.

3.4. ManagedThreadFactory

The `jakarta.enterprise.concurrent.ManagedThreadFactory` allows applications to create thread instances from a Jakarta EE Product Provider without creating new `java.lang.Thread` instances directly. This object allows Application Component Providers to use custom executors such as the `java.util.concurrent.ThreadPoolExecutor` when advanced, specialized execution patterns are required.

Jakarta EE Product Providers can provide custom `Thread` implementations to add management capabilities and container contextual information to the thread.

3.4.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE2.12.2) use a `jakarta.enterprise.concurrent.ManagedThreadFactory` instance to create manageable threads.

The application uses the `jakarta.enterprise.concurrent.ManagedThreadFactoryDefinition` annotation to define instances of `ManagedThreadFactory` and enumerate the required qualifiers for `ManagedThreadFactory` injection points that are to receive a `ManagedThreadFactory` bean that is produced by the `ManagedThreadFactoryDefinition`.

Applications can also retrieve instances using the Java Naming and Directory Interface (JNDI) Naming Context (EE.5) or through injection of resource environment references (EE.5.8.1.1).

The Application Component Provider may use resource environment references to obtain references to a `ManagedThreadFactory` instance as follows:

- Assign an entry in the application component's environment to the reference using the reference type of: `jakarta.enterprise.concurrent.ManagedThreadFactory`. (See EE.5.8.1.2 for information on how resource environment references are declared in the deployment descriptor.)
- This specification recommends, but does not require, that all resource environment references be organized in the appropriate subcontext of the component's environment for the resource type. For Example, all `ManagedThreadFactory` references should be declared in the `java:comp/env/concurrent` subcontext.
- Look up the managed object in the application component's environment using JNDI (EE.5), or through resource injection by the use of the `@Resource` annotation (EE.5.8.1.1).
- New threads are created using the `newThread(Runnable r)` method on the `java.util.concurrent.ThreadFactory` interface.
- The application component thread has permission to interrupt the thread.
- All Threads are contextual (see section 2.3). When the thread is started using the `Thread.start()` method, the `Runnable` that is executed will run with the context of the application component instance that created the `ManagedThreadFactory` instance.



The `ManagedThreadFactory` instance may be invoked from several threads in the application component, each with a different container context (for example, user identity). By always applying the context of the `ManagedThreadFactory` creator, each

thread has a consistent context. If a different context is required for each thread, use the `ContextService` to create a contextual object (see section 3.3).

- If a `ManagedThreadFactory` instance is stopped, all subsequent calls to `newThread()` must throw a `java.lang.IllegalStateException`

3.4.1.1. Usage Example

In this example, an application component uses a background daemon task to dump in-memory events to a database log, similar to the timer usage example in section 3.2.1.1.

The attributes of the `ManagedThreadFactory` reference are documented using the `<description>` tag within the deployment descriptor of the application component and later mapped by the Deployer.

Logger Task

The Logger Task is a long-running task that has the same lifecycle as the servlet. It continually monitors a queue and waits for events to a database log. Its lifecycle is controlled using a `jakarta.servlet.ServletContextListener`.

Resource Environment Reference

The following resource environment reference is added to the `web.xml` file for the web component. The description reflects the desired configuration attributes (see section 3.4.4.2). Alternatively, the `@Resource` annotation can be used in the Servlet code.



Using the description for documenting the configuration attributes of the managed object is optional. The format used here is only an example. Future revisions of Jakarta EE specifications may formalize usages such as this.

```
<resource-env-ref>
  <description>
    This ManagedThreadFactory is used to create a thread for for the
    application's logger task.
    This ManagedThreadFactory has the following requirements:
    Context Info: Local Namespace
  </description>

  <resource-env-ref-name>
    concurrent/LoggerThreadFactory
  </resource-env-ref-name>

  <resource-env-ref-type>
    jakarta.enterprise.concurrent.ManagedThreadFactory
  </resource-env-ref-type>
</resource-env-ref>
```

Task Definition

The task itself simply uses a resource-reference to a JDBC data source, and uses a connect/use/close pattern when invoking the data source.

```
public class LoggerTask implements Runnable {

    DataSource ds = ...;

    public void run() {
        // Wait for data and log it.
        while (!Thread.interrupted()) {
            logEvents(getData(), ds);
        }
    }

    void logEvents(Collection data, DataSource ds) {
        // Iterate through the data and log each row.
        for (...) {
            try (Connection con = ds.getConnection());... {

                // Write the data using our connection.
                ...

                // Commit.
                con.commit();
            }
        }
    }
}
```

Task Submission

The task is started and stopped by a `jakarta.servlet.ServletContextListener`.

```
public class CtxListener implements ServletContextListener {

    Thread loggerThread = null;

    @Resource(name="/concurrent/LoggerThreadFactory")
    ManagedThreadFactory threadFactory;

    public void contextInitialized(ServletContextEvent scEvent) {
        LoggerTask logger = new LoggerTask();
        Thread loggerThread = threadFactory.newThread(logger);
        loggerThread.start();
    }

    public void contextDestroyed(ServletContextEvent scEvent) {
        // Interrupt our logger task since it is no longer available.
    }
}
```

```
// Note: The server will do this for us as well.
if (loggerThread!=null) {
    loggerThread.interrupt();
}
}
```

3.4.2. Application Assembler's Responsibilities

The Application Assembler (EE.2.12.3) is responsible for assembling the application components into a complete Jakarta EE Application and providing assembly instructions that describe the dependencies to the managed objects.

3.4.3. Deployer's Responsibilities

The Deployer (EE.2.12.4) is responsible for deploying the application components into a specific operational environment. In the terms of this specification, the Deployer installs the application components and maps the dependencies defined by the Application Component Provider and Application Assembler to managed objects with the properly defined attributes. See EE.5.8.2 for details.

3.4.4. Jakarta EE Product Provider's Responsibilities

The Jakarta EE Product Provider's responsibilities are as defined in EE.5.8.3 and must support the following:

- Platform threads returned by the `newThread()` method must implement the `ManageableThread` interface. Virtual threads do not implement `ManageableThread`.
- When a `ManagedThreadFactory` instance is stopped, such as when the component that created it is stopped or when the application server is shutting down, all threads that it has created using the `newThread()` method are interrupted. Calls to the `isShutdown()` method in the `ManageableThread` interface on these threads must return true.



The intent is to prevent access to components that are no longer available.

- Threads that are created by a `ManagedThreadFactory` instance but are started after the `ManagedThreadFactory` has shut down is required to start with an interrupted status. Calls to the `isShutdown()` method in the `ManageableThread` interface on these threads must return true.

All threads created by a `ManagedThreadFactory` instance are required to propagate container context information (see section 2.3) to the thread's `Runnable`.

Jakarta EE Product Providers may add any additional container contexts to the managed `ManagedThreadFactory` and provide the means for configuration of those contexts in any way so long as these contexts do not violate the required aspects of this specification.

The following section illustrates some possible configuration options that a Jakarta EE Product Provider may want to provide.

3.4.4.1. ManagedThreadFactory Configuration Attributes

Each managed `ManagedThreadFactory` may support one or more runtime behaviors as specified by configuration attributes. The Jakarta EE Product Provider will determine both the appropriate attributes and the means of configuring those attributes for their product.

3.4.4.2. Configuration Examples

This section and subsections illustrate some examples of how a Jakarta EE Product Provider could configure a `ManagedThreadFactory` and the possible options that such a service could provide.

A `ManagedThreadFactory` can be used directly by application components by using resource environment references, or providers may choose to use the context information supplied as default context propagation policies for `ManagedExecutorService`, or `ManagedScheduledExecutorService` instances. The configuration examples covered in sections 3.1.4.2 and 3.2.4.2 refer to one of the `ManagedThreadFactory` configuration examples that follow.

Each of the examples has the following attributes:

- **Name:** An arbitrary name of the service for the deployer to use as a reference.
- **JNDI name:** The arbitrary, but required, name to identify the service instance. The deployer uses this value to map the service to the component's resource environment reference.
- **Context:** A reference to a `ContextService` instance (see section 3.3). The context service can be used to define the context to propagate to the threads when running tasks. Having multiple `ContextService` instances, each with a different policy, may be desirable for some implementations.
- **Priority:** The priority to assign to the thread (the higher the number, the higher the priority). See the `java.lang.Thread` Javadoc for details on how this value can be used.

Normal Threads

This configuration example illustrates a typical `ManagedThreadFactory` that creates normal priority threads with all available context information.

ManagedThreadFactory	
Name:	Normal Threads
JNDI Name:	Concurrent/tf/normal
Context:	Concurrent/cf/AllContexts
Priority:	5 (Normal)

##Table : Normal ManagedThreadFactory Configuration Example

OLTP Threads

This configuration example describes a `ManagedThreadFactory` that creates threads with a higher than normal priority that can be used for OLTP-type requests.

ManagedThreadFactory	
Name:	OLTP Threads
JNDI Name:	Concurrent/tf/OLTP
Context:	Concurrent/cf/AllContexts
Priority:	6

##Table : OLTP ManagedThreadFactory Configuration Example

Threads for Long-Running Tasks

This configuration example describes a `ManagedThreadFactory` that creates lower-priority threads that can be used for background, long-running tasks.

ManagedThreadFactory	
Name:	Long Running Tasks Threads
JNDI Name:	Concurrent/tf/longRunningThreadsFactory
Context:	Concurrent/cf/AllContexts
Priority:	4

##Table : Long-Running Tasks ManagedThreadFactory Configuration Example

3.4.4.3. Default ManagedThreadFactory

The Jakarta EE Product Provider must provide a preconfigured, default `ManagedThreadFactory` for use by application components under the JNDI name `java:comp/DefaultManagedThreadFactory`. The types of contexts to be propagated by this default `ManagedThreadFactory` from a contextualizing application component must include naming context, class loader, and security information.

The Jakarta EE Product Provider must inject an instance of the default `ManagedThreadFactory` into injection points of `ManagedThreadFactory` that do not have any qualifiers except for where the application provides the producer, in which case the application's producer takes precedence.

3.4.5. System Administrator's Responsibilities

The System Administrator (EE.2.12.5) is responsible for monitoring and overseeing the runtime environment. In the scope of this specification, these duties may include:

- Monitoring for hung tasks.
- Monitoring resource usage (for example, threads and memory).

3.4.6. Transaction Management

`ManagedThreadFactory` implementations must support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions specification with similar semantics to Jakarta Enterprise Beans bean-managed transaction demarcation (see the Jakarta Enterprise Beans specification). User-managed

transactions allow components to manually control global transaction demarcation boundaries. Task implementations may optionally begin, commit, and roll-back a transaction. See EE.4 for details on transaction management in Jakarta EE.

Task instances are run outside of the scope of the transaction of the submitting thread. Any transaction active in the executing thread will be suspended.

3.4.6.1. Jakarta EE Product Provider Requirements

This subsection describes the transaction management requirements of a `ManagedThreadFactory` implementation.

1. The `jakarta.transaction.UserTransaction` interface must be made available in the local JNDI namespace as environment entry: `java:comp/UserTransaction` (EE.5.10 and EE.4.2.1.1)
2. All resource managers must enlist with a `UserTransaction` instance when a transaction is active using the `begin()` method.
3. The executor is responsible for coordinating commits and rollbacks when the transaction ends using `commit()` and `rollback()` methods.
4. A task must have the same ability to use transactions as the component submitting the tasks. For example, tasks are allowed to call transactional enterprise beans, and managed beans that use the `@Transactional` interceptor as defined in the Jakarta Transactions specification.

3.4.6.2. Application Component Provider's Requirements

This subsection describes the transaction management requirements of each task provider's implementation.

1. A task instance that starts a transaction must complete the transaction before starting a new transaction.
2. The task provider uses the `jakarta.transaction.UserTransaction` interface to demarcate transactions.
3. Transactions are demarcated using the `begin()`, `commit()`, and `rollback()` methods of the `UserTransaction` interface.
4. While an instance is in an active transaction, resource-specific transaction demarcation APIs must not be used (e.g. if a `java.sql.Connection` is enlisted in the transaction instance, the `Connection.commit()` and `Connection.rollback()` methods must not be used).
5. The task instance must complete the transaction before the task method ends.

See the example titled *UserTransaction Usage Example* under section 3.1.8.2 for an example of using a `UserTransaction` within a task.

3.5. Beans for Managed Objects

The resource definition annotations, `ContextServiceDefinition`, `ManagedExecutorDefinition`, `ManagedScheduledExecutorDefinition`, and `ManagedThreadFactoryDefinition`, provide a `qualifiers` attribute that the application can optionally configure to specify a list of required qualifier

annotation classes. Similarly, the corresponding deployment descriptor elements, `<context-service>`, `<managed-executor>`, `<managed-scheduled-executor>`, and `<managed-thread-factory>`, allow `<qualifier>` sub-elements that the application can optionally configure to specify one or more required qualifier annotation class names.

When a non-empty list of qualifier classes is configured for a resource definition, the container creates an instance of the corresponding managed object and registers a `jakarta.enterprise.context.ApplicationScoped` bean for it with the configured qualifiers.

The container also creates a default instance of each managed object type for which the application does not already produce a bean without required qualifiers, registering a `jakarta.enterprise.context.ApplicationScoped` bean without qualifiers for the default instance.

The life cycle of these beans aligns with the life cycle of the application component. The configured qualifier annotation classes must be accessible to the application and must not have any attributes without default values. The application must not configure `java:global` names on resource definitions that have a non-empty list of qualifier classes. The container raises an error and does not register the bean if these requirements are not met.

3.5.1. Qualifiers Example

In this example, the application configures a single qualifier annotation when defining a `ManagedExecutorService`. The application injects the `ManagedExecutorService` bean by specifying the qualifier annotation on the injection point.

3.5.1.1. Example Qualifier Annotation Class

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import jakarta.inject.Qualifier;

@Qualifier
@Retention(RetentionPolicy.RUNTIME)
@Target({ ElementType.FIELD, ElementType.METHOD, ElementType.PARAMETER, ElementType
    .TYPE })
public @interface MaxAsync5 {
}
```

3.5.1.2. Example usage of Qualifier Annotation

```
@ManagedExecutorDefinition(
    name = "java:comp/concurrent/MyExecutor",
    qualifiers = MaxAsync5.class,
    maxAsync = 5)
public class MyServlet extends HttpServlet {
```

```
@Inject
@Max5Async
ManagedExecutorService executor;

...
}
```

Chapter 4. Thread Context Providers

The `ThreadContextProvider` SPI standardizes the interaction between third-party providers of thread context and the Jakarta EE Product Provider, enabling third-party thread context types to be captured and propagated alongside the Jakarta EE Product Provider's built-in thread context types.

This is useful for the model where a Jakarta EE Product Provider pieces together various third-party artifacts, together with its own, to produce a Jakarta EE compliant product.

A `ThreadContextProvider` indicates its provided thread context type, provides a way to capture snapshots of that thread context type, provides a way for applying empty/cleared context of that type to threads, and provides a way to restore the previous thread context after a contextual task or action completes.

4.1. Responsibilities and Requirements

4.1.1. Third-Party Context Provider's Requirements

This subsection describes the requirements of a third-party provider of thread context.

1. Implement the `jakarta.enterprise.concurrent.spi.ThreadContextProvider` interface.
2. Include a file with location and name of `META-INF/services/jakarta.enterprise.concurrent.spi.ThreadContextProvider` within its JAR file. The content of this file must be one or more lines, each specifying the fully qualified name of a `ThreadContextProvider` implementation that is provided within the JAR file.
3. Capture or produce snapshots of the provided type of thread context when `ThreadContextProvider` methods are invoked.
4. Apply a snapshot of the provided type of thread context to a thread, upon request from the Jakarta EE Product Provider.
5. Remove a snapshot of the provided type of thread context from the thread, restoring the previous thread context, upon request from the Jakarta EE Product Provider.

4.1.2. Jakarta EE Product Provider's Requirements

This subsection describes the requirements of the Jakarta EE Product Provider.

1. Use the `java.util.ServiceLoader` to load available `jakarta.enterprise.concurrent.spi.ThreadContextProvider` implementations from the thread context class loader.
2. Read configuration, which could be from a `jakarta.enterprise.ContextServiceDefinition` or a vendor-specific configuration mechanism, to identify which context types to propagate, clear, and ignore.
3. Invoke the `ThreadContextProvider.currentContext` method to capture context that is configured to be propagated from the current thread to the thread that will run the contextual task or action.

4. Invoke the `ThreadContextProvider.clearedContext` method to generate a snapshot of context that is configured to be cleared from the thread that will run the contextual task or action.
5. Invoke the `ThreadContextSnapshot.begin` method to establish context on a thread that will run the contextual task or action.
6. Invoke the `ThreadContextRestorer.endContext` method to restore the previous context after the contextual task or action completes.

4.2. Usage Examples

4.2.1. Custom Thread Context Example

This example supplies a `ThreadContextProvider` that turns priority (from `java.lang.Thread`) into a third-party context type. Although not useful in practice (it is better to let priority be managed by the managed executor service), this example is chosen because the concept of thread priority is simple, well understood, and already built into Java, allowing the reader to focus on the mechanisms of thread context capture/propagation/restore rather than the details of the context type itself.

4.2.1.1. Example of Custom ThreadContextProvider

The interface, `jakarta.enterprise.concurrent.spi.ThreadContextProvider`, is the means by which the Jakarta EE Product Provider requests the capturing of a particular context type from the current thread. It also provides a way to obtain a snapshot of empty/cleared context of this type and identifies the name by which the user refers to this context type when configuring a `ContextServiceDefinition`.

```
package example.jakarta.concurrency.priority;

import jakarta.enterprise.concurrent.spi.ThreadContextProvider;
import jakarta.enterprise.concurrent.spi.ThreadContextSnapshot;
import java.util.Map;

public class ThreadPriorityContextProvider implements ThreadContextProvider {
    public String getThreadContextType() {
        return "ThreadPriority";
    }

    public ThreadContextSnapshot currentContext(Map<String, String> props) {
        return new ThreadPrioritySnapshot(Thread.currentThread().getPriority());
    }

    public ThreadContextSnapshot clearedContext(Map<String, String> props) {
        return new ThreadPrioritySnapshot(Thread.NORM_PRIORITY);
    }
}
```

4.2.1.2. Example of Custom ThreadContextSnapshot and ThreadContextRestorer

The interface, `jakarta.enterprise.concurrent.spi.ThreadContextSnapshot`, represents a snapshot of thread context. The Jakarta EE Product Provider can request the context represented by this snapshot be applied to any number of threads by invoking the `begin` method. An instance of `jakarta.enterprise.concurrent.spi.ThreadContextRestorer`, which is returned by the `begin` method, stores the previous context of the thread. The `ThreadContextRestorer` instance is provided for one-time use by the Jakarta EE Product Provider to restore the previous context after the context represented by the snapshot is no longer needed on the thread.

```
package example.jakarta.concurrency.priority;

import jakarta.enterprise.concurrent.spi.ThreadContextRestorer;
import jakarta.enterprise.concurrent.spi.ThreadContextSnapshot;
import java.util.concurrent.atomic.AtomicBoolean;

public class ThreadPrioritySnapshot implements ThreadContextSnapshot {
    final int priority;

    ThreadPrioritySnapshot(int priority) {
        this.priority = priority;
    }

    public ThreadContextRestorer begin() {
        Thread thread = Thread.currentThread();
        int priorityToRestore = thread.getPriority();
        AtomicBoolean restored = new AtomicBoolean();

        ThreadContextRestorer contextRestorer = () -> {
            if (restored.compareAndSet(false, true))
                thread.setPriority(priorityToRestore);
            else
                throw new IllegalStateException();
        };

        thread.setPriority(priority);

        return contextRestorer;
    }
}
```

4.2.1.3. ServiceLoader Entry

To make the `ThreadContextProvider` implementation available to the `ServiceLoader`, the provider JAR includes a file of the following name and location,

```
META-INF/services/jakarta.enterprise.concurrent.spi.ThreadContextProvider
```

which contains the following line,

```
example.jakarta.concurrency.priority.ThreadPriorityContextProvider
```

4.2.1.4. Usage of the Custom Context Type from a Servlet

The following example shows application code that uses a `ManagedExecutorService` that propagates the example context type. If the provider is implemented correctly and made available on the application's thread context class loader, the async `Runnable` should report that it is running with a priority of 3.

```
import jakarta.annotation.Resource;
import jakarta.enterprise.concurrent.ContextServiceDefinition;
import jakarta.enterprise.concurrent.ManagedExecutorDefinition;
import jakarta.enterprise.concurrent.ManagedExecutorService;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@ContextServiceDefinition(
    name = "java:module/concurrent/PriorityContext",
    propagated = "ThreadPriority")
@ManagedExecutorDefinition(
    name = "java:module/concurrent/PriorityExec",
    context = "java:module/concurrent/PriorityContext")
public class MyServlet extends HttpServlet {
    @Resource(lookup = "java:module/concurrent/PriorityExec")
    ManagedExecutorService executor;

    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        Thread.currentThread().setPriority(3);

        executor.runAsync(() -> {
            System.out.println("Running with priority of " +
                Thread.currentThread().getPriority());
        });
    }
}
```

Chapter 5. Asynchronous Methods

The `jakarta.enterprise.concurrent.Asynchronous` annotation annotates a CDI managed bean method to run asynchronously. The CDI managed bean must not be a Jakarta Enterprise Bean, and neither the method nor its class can be annotated with the MicroProfile Asynchronous annotation.

Each invocation of an asynchronous method by the user corresponds to a managed `java.util.concurrent.CompletableFuture` instance that is backed by a `jakarta.enterprise.concurrent.ManagedExecutorService` as its default asynchronous execution facility. Its dependent stages (and all dependent stages that are created from those, and so on) continue to be backed by the managed executor service, which also manages the propagation of context to completion stage actions. Application code, including from within the asynchronous method, can query the status of the completable future instance and can choose to complete it at any time and by any means.

5.1. Scheduled Asynchronous Methods

The `runAt` attribute of `Asynchronous` allows schedules to be assigned to asynchronous methods, such that the `java.util.concurrent.CompletableFuture` for the asynchronous method represents the completion of all executions in the schedule. With each execution, the method indicates that subsequent executions are needed by returning a `null` result value. The Jakarta EE Product attempts to run the method at its scheduled times until its future is completed or the method returns a non-null result value or raises an exception.

5.2. Responsibilities and Requirements

5.2.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE2.12.2) specify the `jakarta.enterprise.concurrent.Asynchronous` annotation on CDI managed bean methods with return type of `java.util.concurrent.CompletableFuture`, `java.util.concurrent.CompletionStage`, or `void` to designate them for asynchronous execution.

The Application Component Provider supplies the implementation of the asynchronous method. If the method has return type of `java.util.concurrent.CompletableFuture` or `java.util.concurrent.CompletionStage`, its implementation arranges for the completion of a completable future or completion stage, which it returns as the method result. The asynchronous method can create or obtain a new completion stage for this purpose, or it can use the `jakarta.enterprise.concurrent.Asynchronous.Result` API to obtain the same instance that is being returned to the caller of the asynchronous method.

5.2.1.1. Usage Example

In this example, an application component wants to asynchronously identify similar items that a customer might wish to purchase.

Asynchronous Method Definition

To check if the recommended similar items are currently available for purchase, this asynchronous method relies on an external database that is accessible via a resource reference that is defined in the application component's `java:comp` namespace, which must be made available to the asynchronous method.

```
public class ProductRecommendations {
    @Asynchronous(executor = "java:module/env/concurrent/myExecutorRef")
    public CompletableFuture<Set<Item>> findSimilar(Cart cart, History h) {
        Set<Item> combined = new LinkedHashSet<Item>();
        for (Item item : cart.items())
            combined.addAll(item.similar());
        for (Item item : h.recentlyViewed(3))
            combined.addAll(item.similar());
        combined.removeAll(cart.items());

        try (Connection con = ((DataSource) InitialContext.doLookup(
            "java:comp/env/jdbc/ds1")).getConnection()) {
            PreparedStatement stmt = con.prepareStatement(CHECK_AVAILABILITY);
            for (Item item : combined) {
                ... Remove if the similar item is unavailable
            }
        } catch (NamingException | SQLException x) {
            throw new CompletionException(x);
        }
        return Asynchronous.Result.complete(combined);
    }
}
```

Asynchronous Method Invocation

The CDI managed bean with the asynchronous method is injected into a `Servlet`, which uses it to asynchronously determine the product recommendations.

```
public class CheckoutServlet extends HttpServlet {
    @Inject
    ProductRecommendations recommendations;

    @Resource(name="java:comp/env/jdbc/ds1", lookup="jdbc/ds1")
    DataSource ds;

    public void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException {
        ...
        recommendations.findSimilar(cust.getCart(), cust.getHistory())
            .thenAccept(recommended -> {
                ... Update page with recommendations
            });
    }
}
```

```
    ...  
  }  
}
```

5.2.2. Jakarta EE Product Provider's Responsibilities

The Jakarta EE Product Provider's responsibilities are as defined in EE.5.8.3.

The Jakarta EE Product Provider registers a CDI interceptor to arrange for the invocation of asynchronous methods on the `jakarta.enterprise.concurrent.ManagedExecutorService` or `jakarta.enterprise.concurrent.ManagedScheduledExecutorService` that is specified by the `jakarta.enterprise.concurrent.Asynchronous` annotation.

The Jakarta EE Product Provider creates a `java.util.concurrent.CompletableFuture` instance to associate with each asynchronous method invocation, returning this same instance to the caller of the asynchronous method, and providing it to the asynchronous method implementation by means of the `jakarta.enterprise.concurrent.Asynchronous.Result` API. The Jakarta EE Product Provider completes this instance upon completion of the completion stage that is returned by the asynchronous method implementation, which is a no-op when the asynchronous method implementation chooses to return the same instance. If the asynchronous method return type is `void` and the asynchronous method does not have a schedule, or if the asynchronous method implementation raises an error or exception, the Jakarta EE Product Provider completes this instance upon return from the asynchronous method implementation. The Jakarta EE Product Provider raises `java.util.concurrent.RejectedExecutionException` to the caller of the asynchronous method if it cannot accept a method for asynchronous execution, for example if supplied with an invalid JNDI name. If the Jakarta EE Product Provider cannot start the asynchronous method for any reason after this point, it completes the `java.util.concurrent.CompletableFuture` instance with a `java.util.concurrent.CancellationException`.

5.3. Transaction Management

When an asynchronous method is also annotated with `jakarta.transaction.Transactional`, the transactional types which can be used are:

- `jakarta.transaction.Transactional.TxType.REQUIRES_NEW` - which causes the method to run in a new transaction
- `jakarta.transaction.Transactional.TxType.NOT_SUPPORTED` - which causes the method to run with no transaction

All other transaction attributes must result in `java.lang.UnsupportedOperationException` upon invocation of the asynchronous method.

When an asynchronous method is not annotated as `jakarta.transaction.Transactional` or the transaction type is set to `TxType.NOT_SUPPORTED`, the Jakarta EE Product Provider must support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions specification. User-managed transactions allow components to manually control global transaction demarcation boundaries. Task implementations may optionally begin, commit, and roll-back a transaction. See EE.4 for details on transaction

management in Jakarta EE.

Chapter 6. Scheduled Methods

The `jakarta.enterprise.concurrent.Schedule` annotation can annotate a CDI managed bean method to cause the method to run automatically on a schedule. The method must have a `void` return type and no parameters. The bean must not be a Jakarta Enterprise Bean.

After the application starts, the Jakarta EE Product Provider arranges for non-overlapping invocation of the scheduled method to run on the default `ManagedScheduledExecutorService` according to the schedule.

6.1. Skipped Invocation

A scheduled time is skipped if the difference between its actual start time and its target start time would exceed the `skipIfLateBy` field of the `Schedule` annotation or if the prior invocation ended after its scheduled time.

6.2. Lifecycle

Once the method runs, or if late, is skipped, the completion time (or time of the skip) is used to compute the next time according to the schedule. Invocation of the method continues according to the scheduled times until the application stops or an invocation of the method raises an error or exception, after which no further invocation of the method is scheduled.

6.3. Responsibilities and Requirements

6.3.1. Application Component Provider's Responsibilities

Application Component Providers (application developers) (EE2.12.2) specify the `jakarta.enterprise.concurrent.Schedule` annotation on one or more CDI managed bean methods that have a return type of `void` and no parameters.

The Application Component Provider supplies the implementation of the scheduled method. The application may raise an exception from the method to prevent subsequent invocations of the method from continuing to be scheduled.

The application must not invoke the method directly or submit it for invocation in any other way.

6.3.1.1. Usage Example

In this example, an application component wants to run some business logic every Monday, Wednesday, and Friday at 10:00 AM Central US time.

```
@ApplicationScoped
public class ScheduleBean {
    @Schedule(daysOfWeek = { DayOfWeek.MONDAY, DayOfWeek.WEDNESDAY, DayOfWeek.FRIDAY },
        hours = 10,
        zone = "America/Chicago")
```

```
public void monWedFriAt10() {  
    System.out.println("Running at " + ZonedDateTime.now());  
}  
}
```

6.3.2. Jakarta EE Product Provider's Responsibilities

The Jakarta EE Product Provider's responsibilities are as defined in EE.5.8.3.

The Jakarta EE Product Provider identifies CDI managed bean methods that are annotated with the `jakarta.enterprise.concurrent.Schedule` annotation and computes the next time from the schedule.

The Jakarta EE Product Provider submits tasks to the default `ManagedScheduledExecutorService` to invoke scheduled methods, compute the next time, and to reschedule for that time.

The Jakarta EE Product Provider makes the component context available on the thread that invokes the scheduled method, such that the scheduled method is able to look up resources in the component namespace.

The Jakarta EE Product Provider ceases to invoke scheduled methods after the method raises an error or exception or the application stops.

6.4. Transaction Management

When a scheduled method is also annotated with `jakarta.transaction.Transactional`, the transactional types which can be used are:

- `jakarta.transaction.Transactional.TxType.REQUIRES_NEW` - which causes the method to run in a new transaction
- `jakarta.transaction.Transactional.TxType.NOT_SUPPORTED` - which causes the method to run with no transaction

When a scheduled method is not annotated as `jakarta.transaction.Transactional` or the transaction type is set to `TxType.NOT_SUPPORTED`, the Jakarta EE Product Provider must support user-managed global transaction demarcation using the `jakarta.transaction.UserTransaction` interface, which is described in the Jakarta Transactions specification. User-managed transactions allow components to manually control global transaction demarcation boundaries. Scheduled method implementations may optionally begin, commit, and roll back a transaction. See EE.4 for details on transaction management in Jakarta EE.